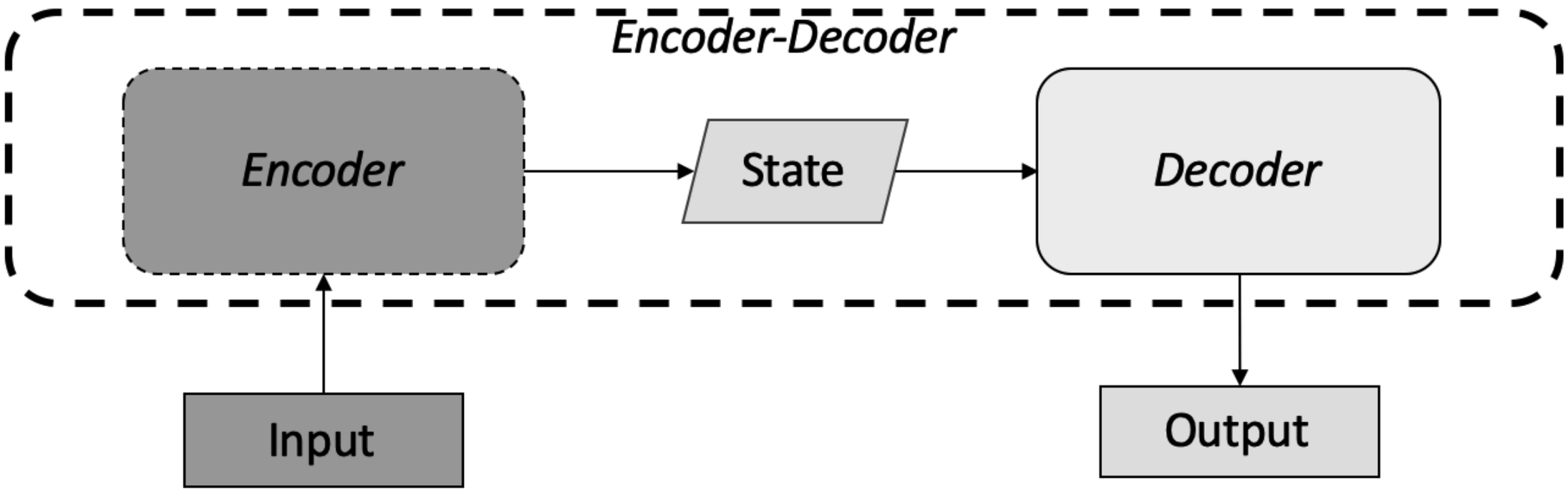
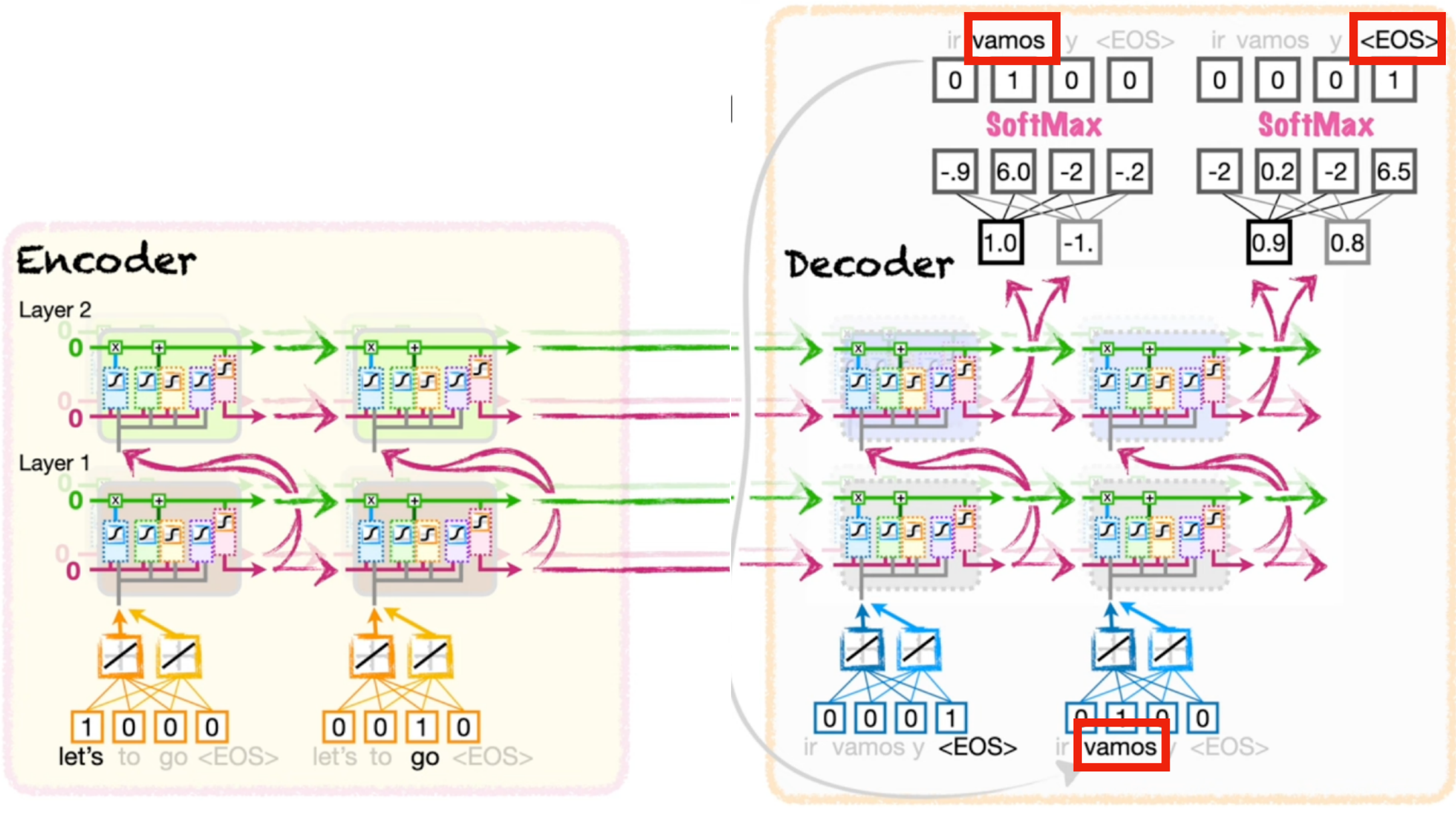
Transformers

Encoder-Decoder



Example


Encoder
Layer 2
Layer 1
let's to go <EOS>
let's to go <EOS>
Decoder
ir vamos y <EOS>
ir vamos y <EOS>
SoftMax
SoftMax
ir vamos y <EOS>
ir vamos y <EOS>

Encoder Decoder

- The basics are easy to follow once we know RNN

- They are big models with some issues on Scalability and some training issues (exploding gradient!)

- However, they are the corner stones to the latest models: Transformers, GPT and others…

Transformers

Timeline

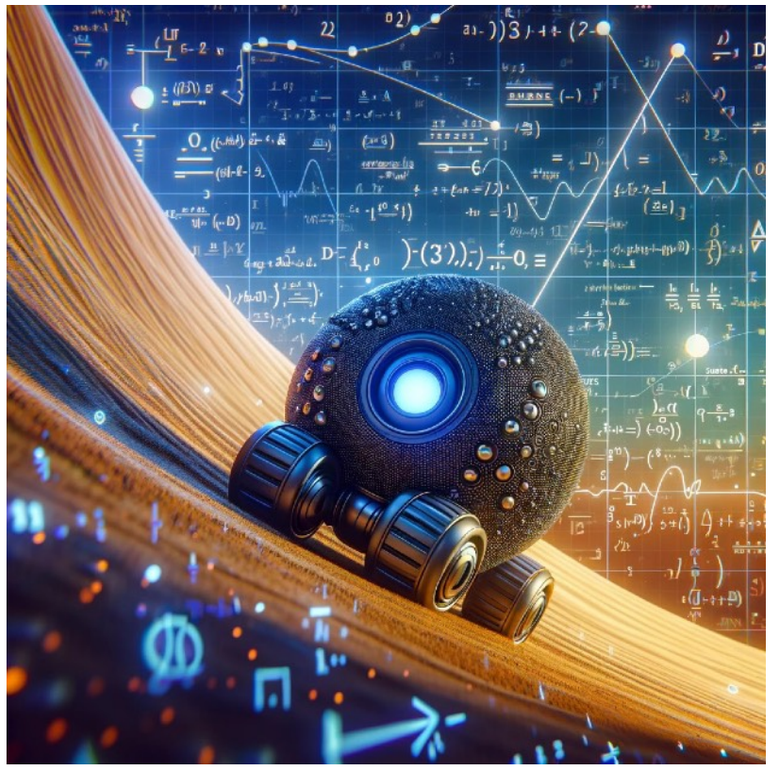
- 1990s - RNNs, LSTMS
- 2014 - Attention Mechanism (RNNs)
- 2017 - Transformer - “Attention is all you need”
- 2018 - NLP taken by storm: BERT, GPT-3 (LLMs start to emerge)
- …,2020 - Vision transformer (ViT), Alphafold-2 (RL)
- 2021 - (ML) Generative AI Era - GPT-X, DALL-E
- 2022 - Chat-GPT, Robotics Transformer, Stable Diffusion
- 2024 - Multimodal models, GPT-4, Text to Video (Sora, Dinov2), Devon
- 2025 - Claude, DeepSeek, Ernie (fresh! March 2025)

Starting Applications

- NLP tasks
    - Translation, classification, summarisation, spelling and grammar
    - Time-series data
    - …
    - But actually… they are now everywhere!

Only one Modality?

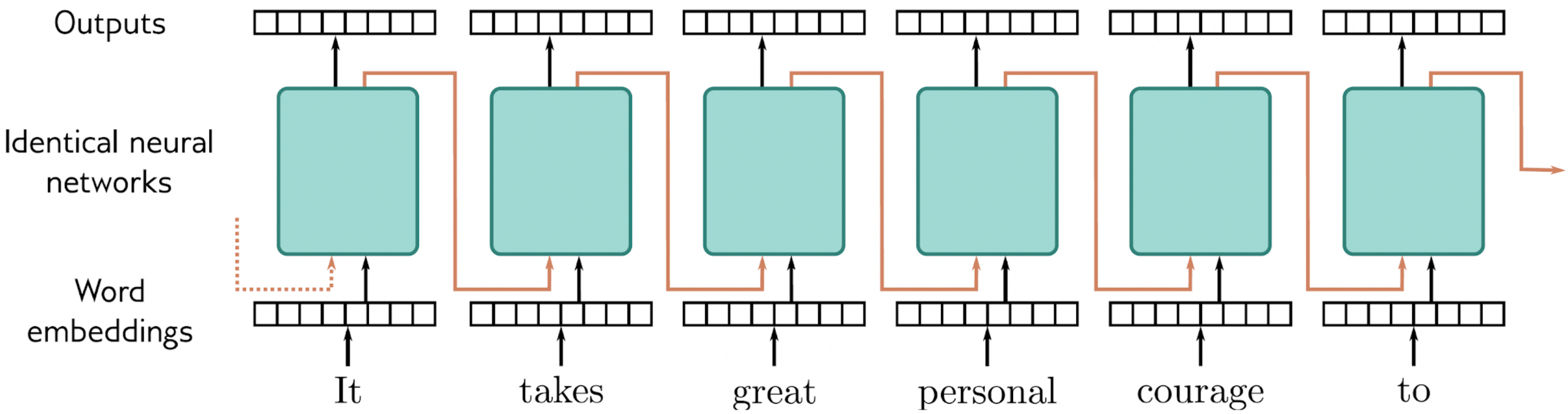
- For starters, DALL-E - text to image based on transformers



Motivation

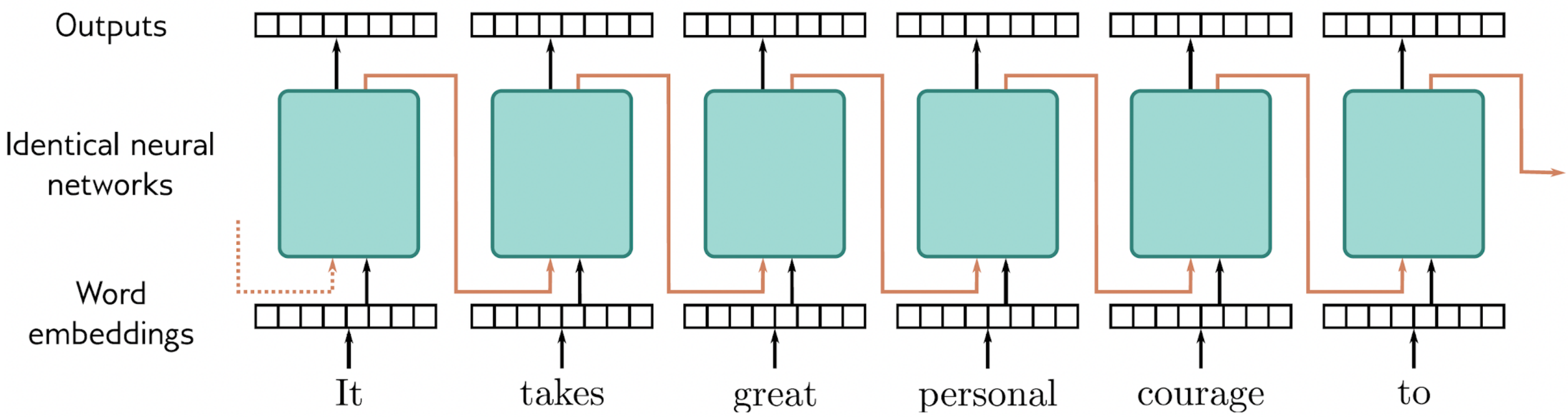
- Encoder Decoder presented a good solution to seq2seq at the time
    - They are heavy (remember original paper 380M in 2018)

More Problems?



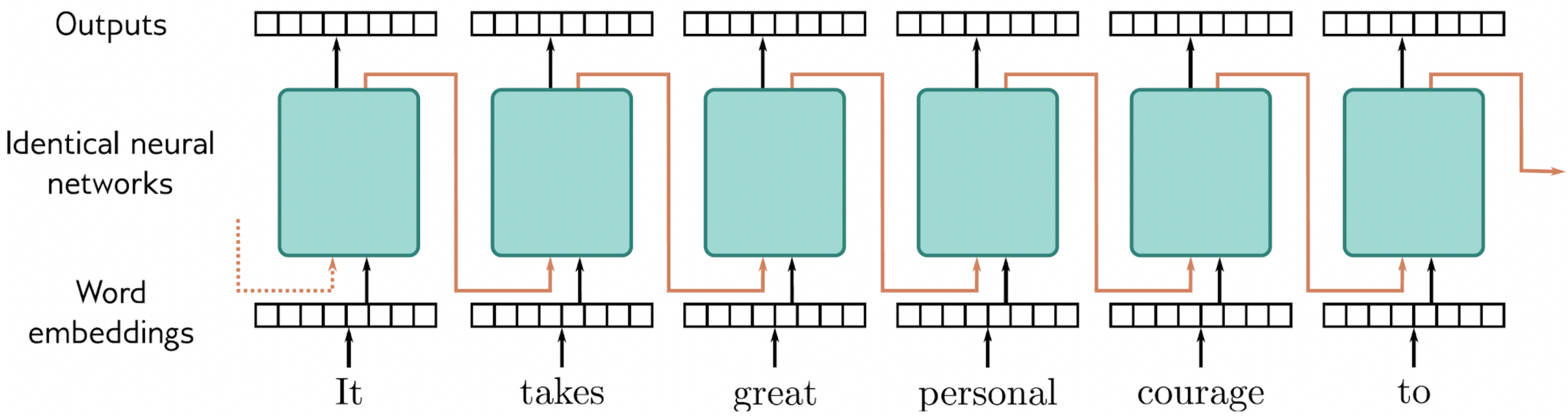
Motivation

- Encoder Decoder presented a good solution to seq2seq at the time
  - The RNN based version is recurrent - h_{t+1} depends on h_t - **not parallelizable**!



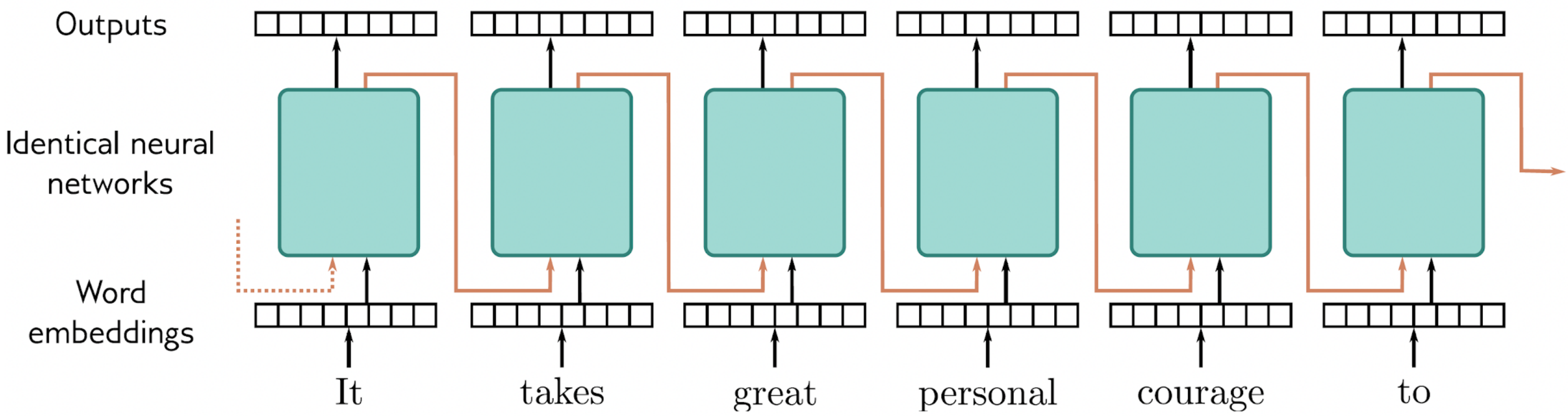
Motivation

- Encoder Decoder presented a good solution to seq2seq at the time
    - Authors claimed the need to limit the signal paths required for long-range dependencies



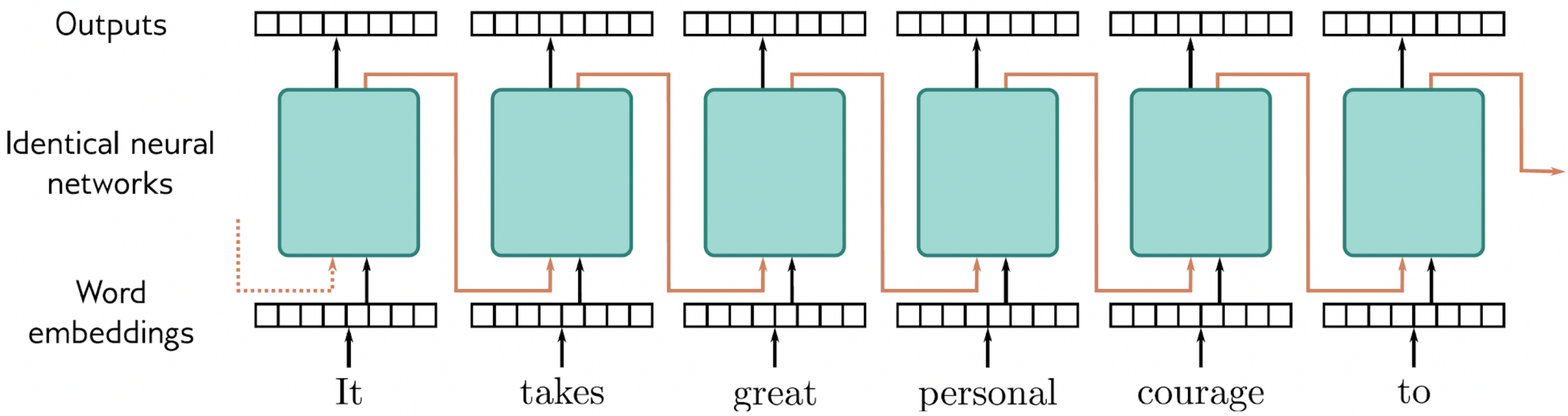
Motivation

- Encoder Decoder presented a good solution to seq2seq at the time
    - RNNs, scale linearly with the input sequence, and tend to forget, words and context!



Motivation

- Encoder Decoder presented a good solution to seq2seq at the time
    - LSTMs alleviate the issue but take long time to train! (Still sequence processing!)



The Transformer Network

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

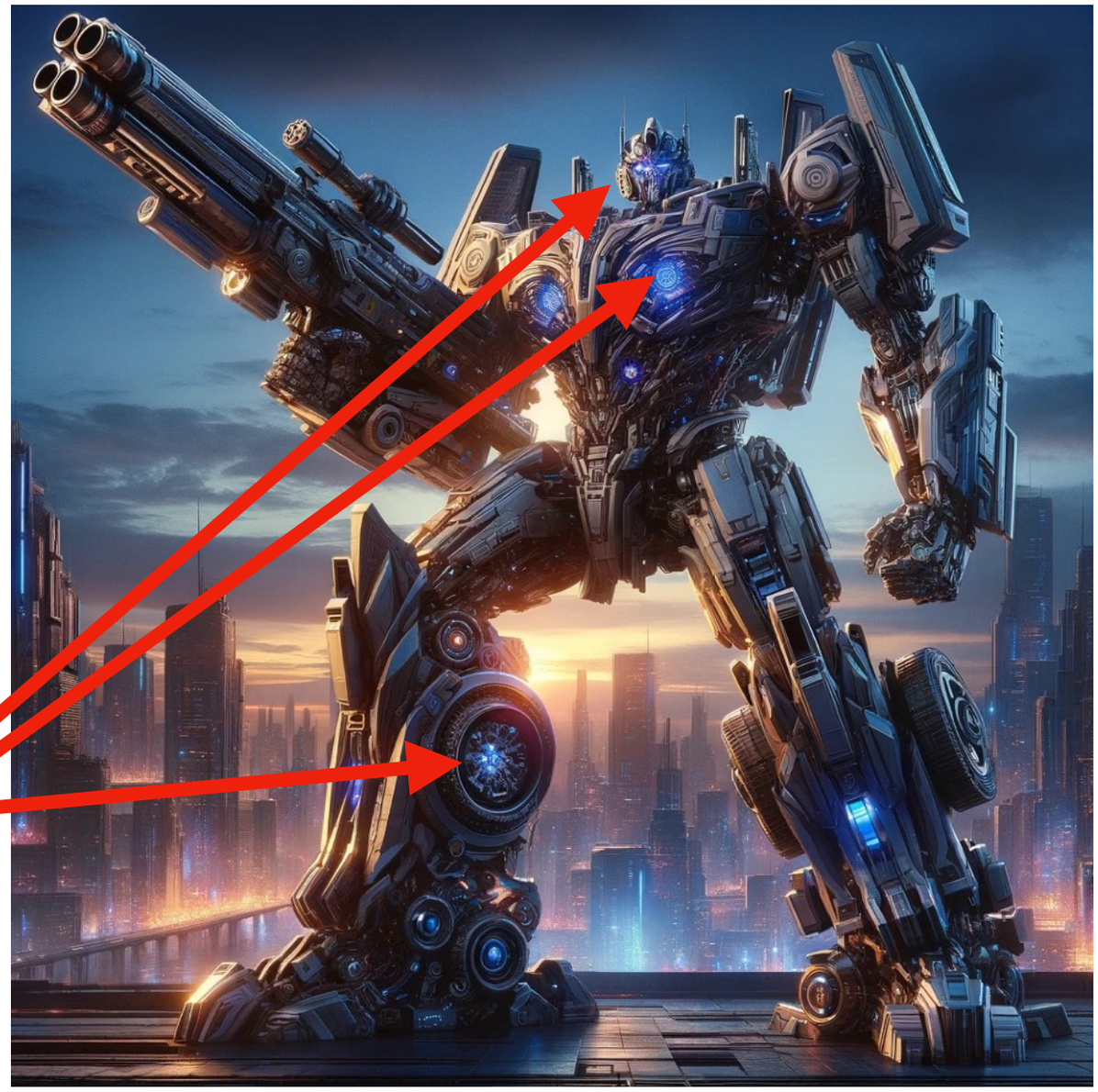
The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Anatomy of a Transformer

Anatomy of a Transformer

Huge model !

Anatomy of a Transformer

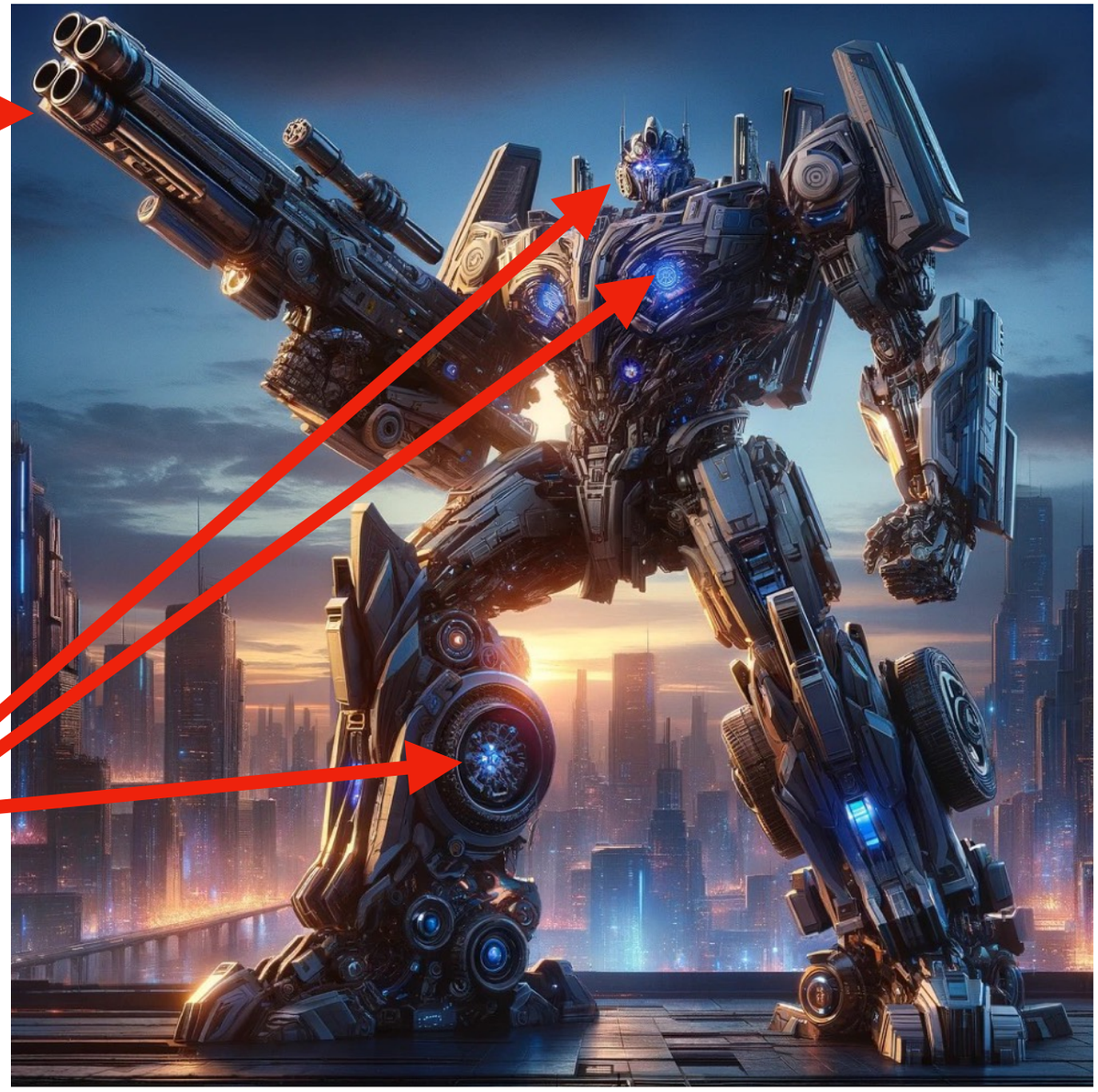


Lot of input energy to function

Huge model !

Anatomy of a Transformer

Big guns

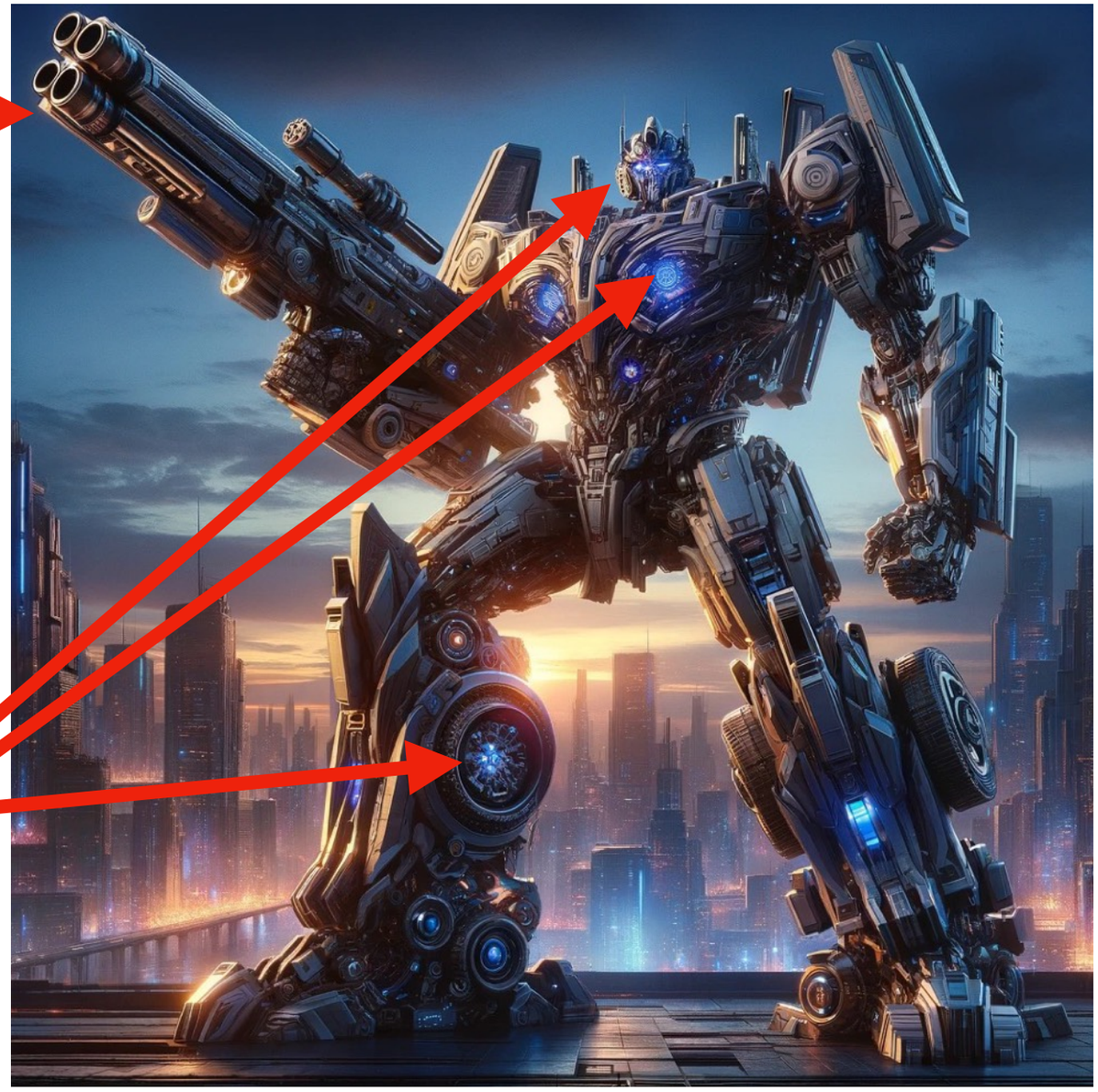


Lot of input energy to function

Huge model !

Anatomy of a Transformer

Big guns

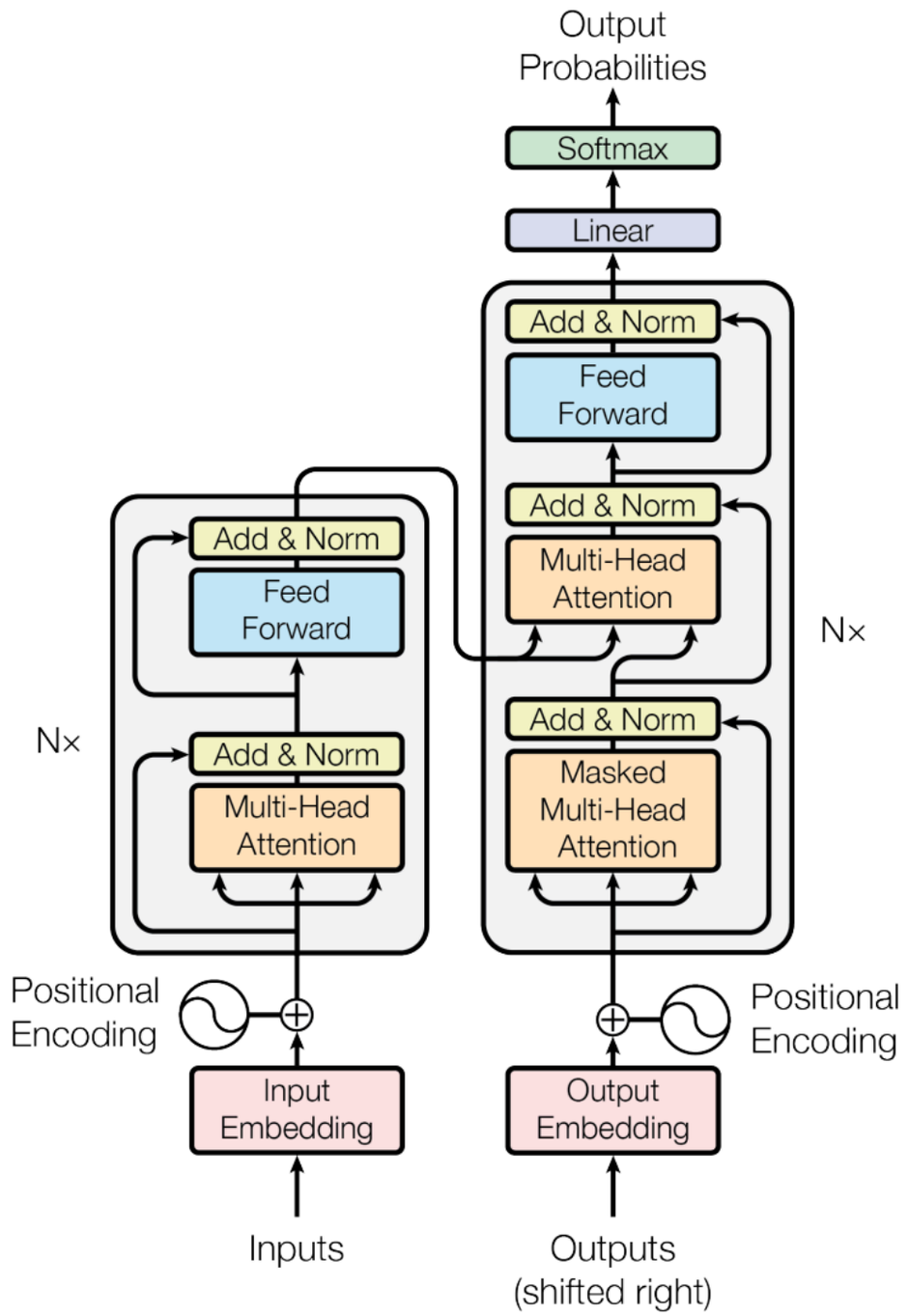


Huge model !

Lot of input energy to function

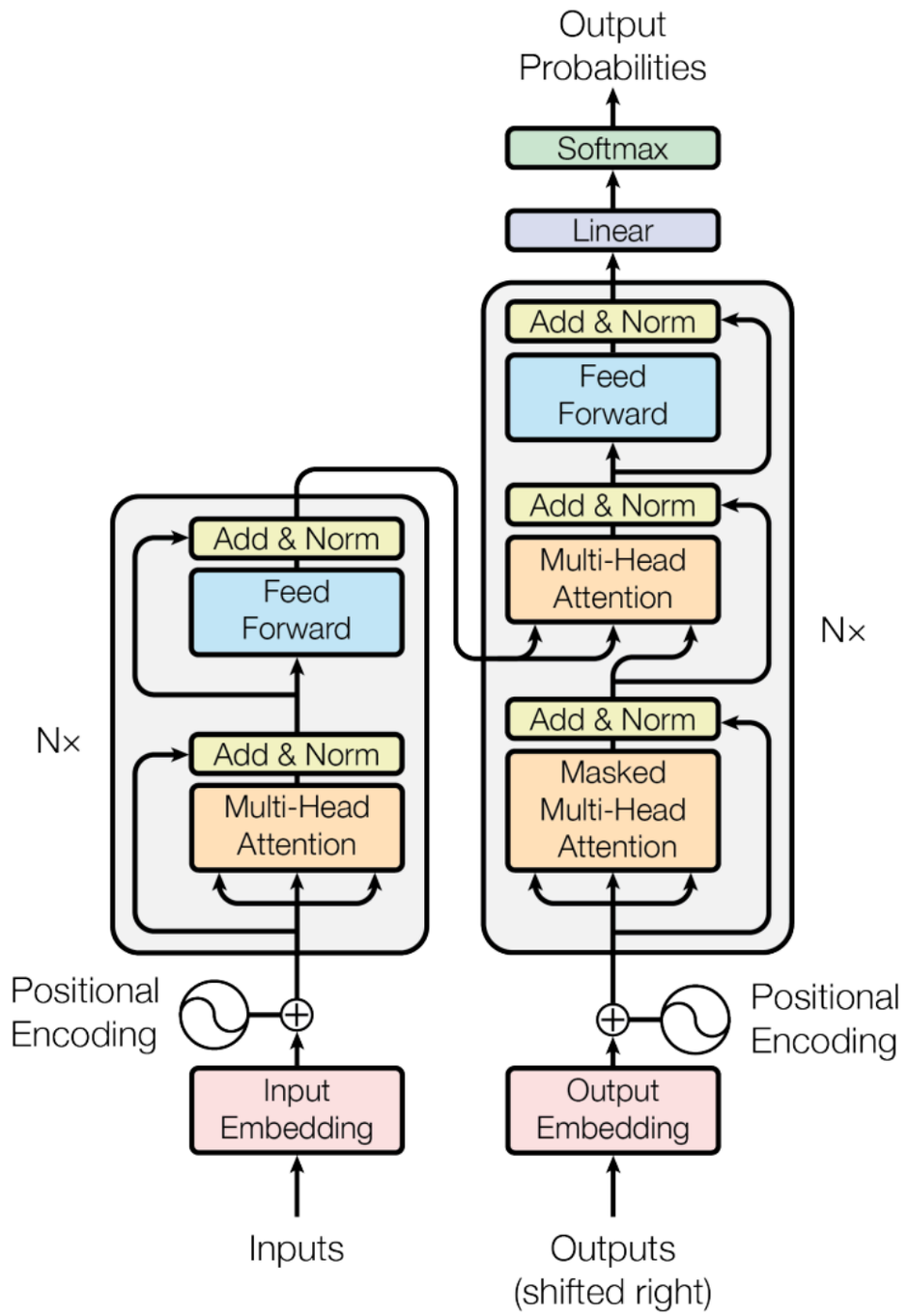
Its amazing!

Anatomy of a Transformer



Anatomy of a Transformer

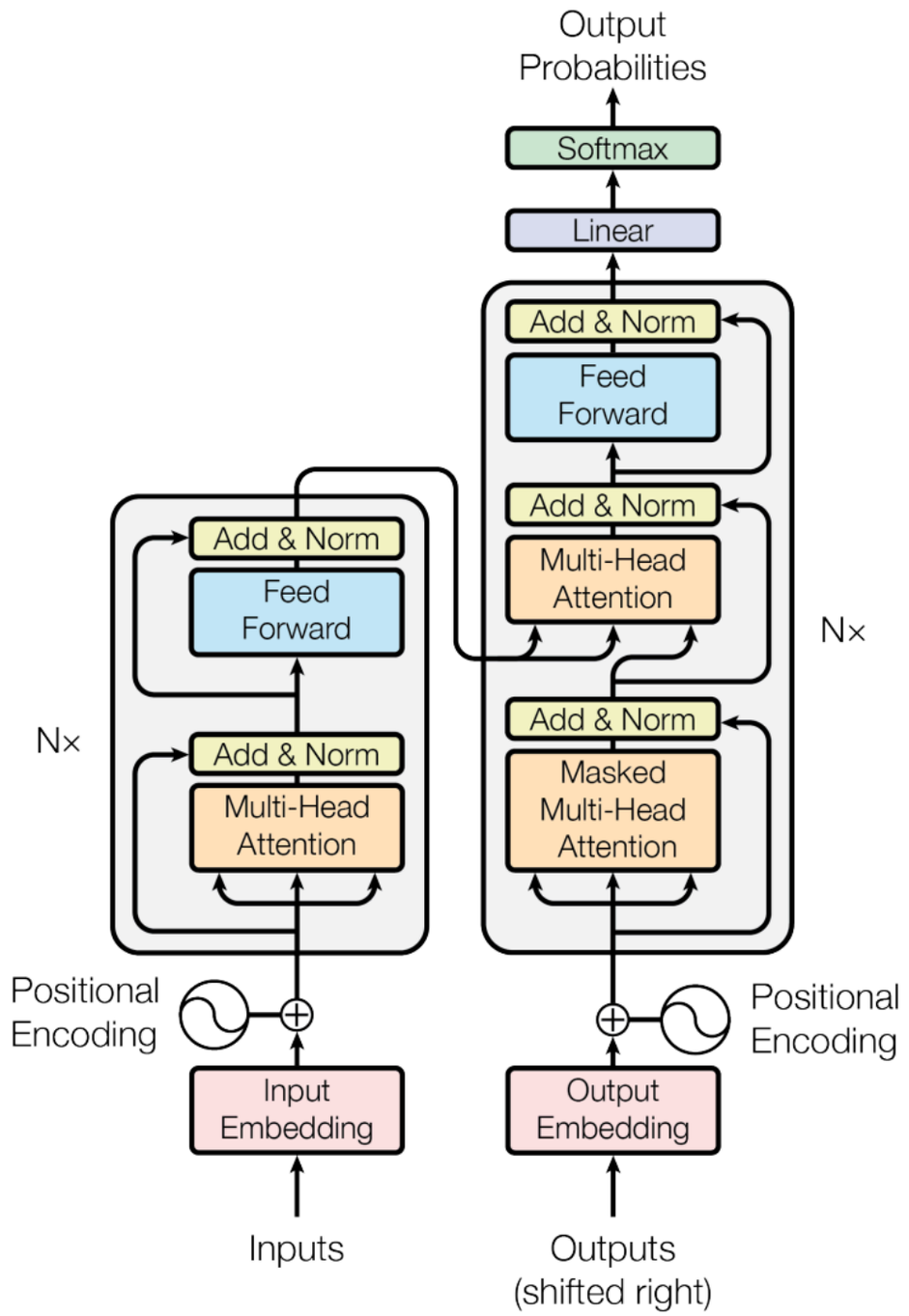
- Huge models and requires a lot of input data it is not new, we already had models bigger than this!



Anatomy of a Transformer

- Huge models and requires a lot of input data it is not new, we already had models bigger than this!

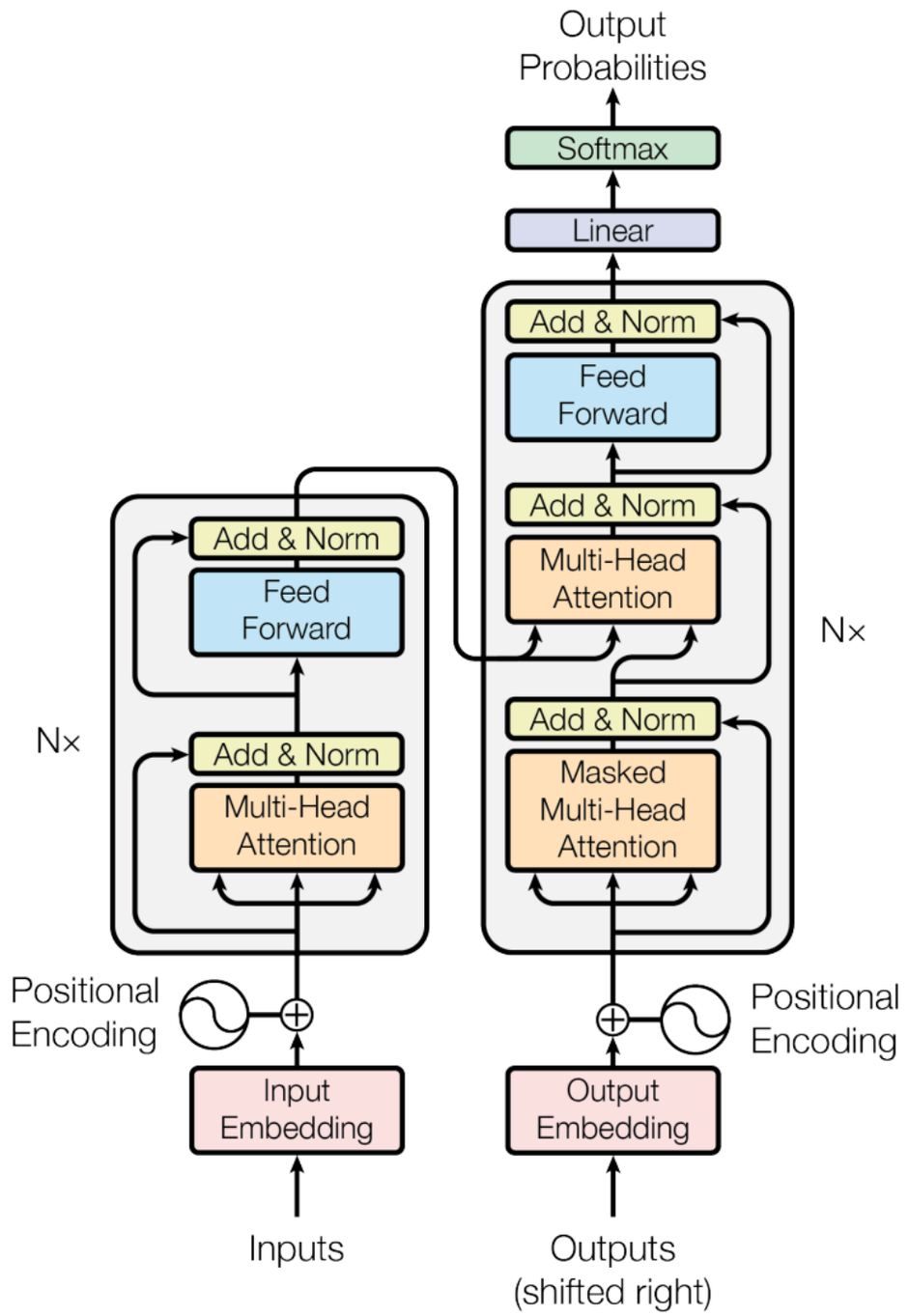
- Big guns ?



Anatomy of a Transformer

- Huge models and requires a lot of input data it is not new, we already had models bigger than this!

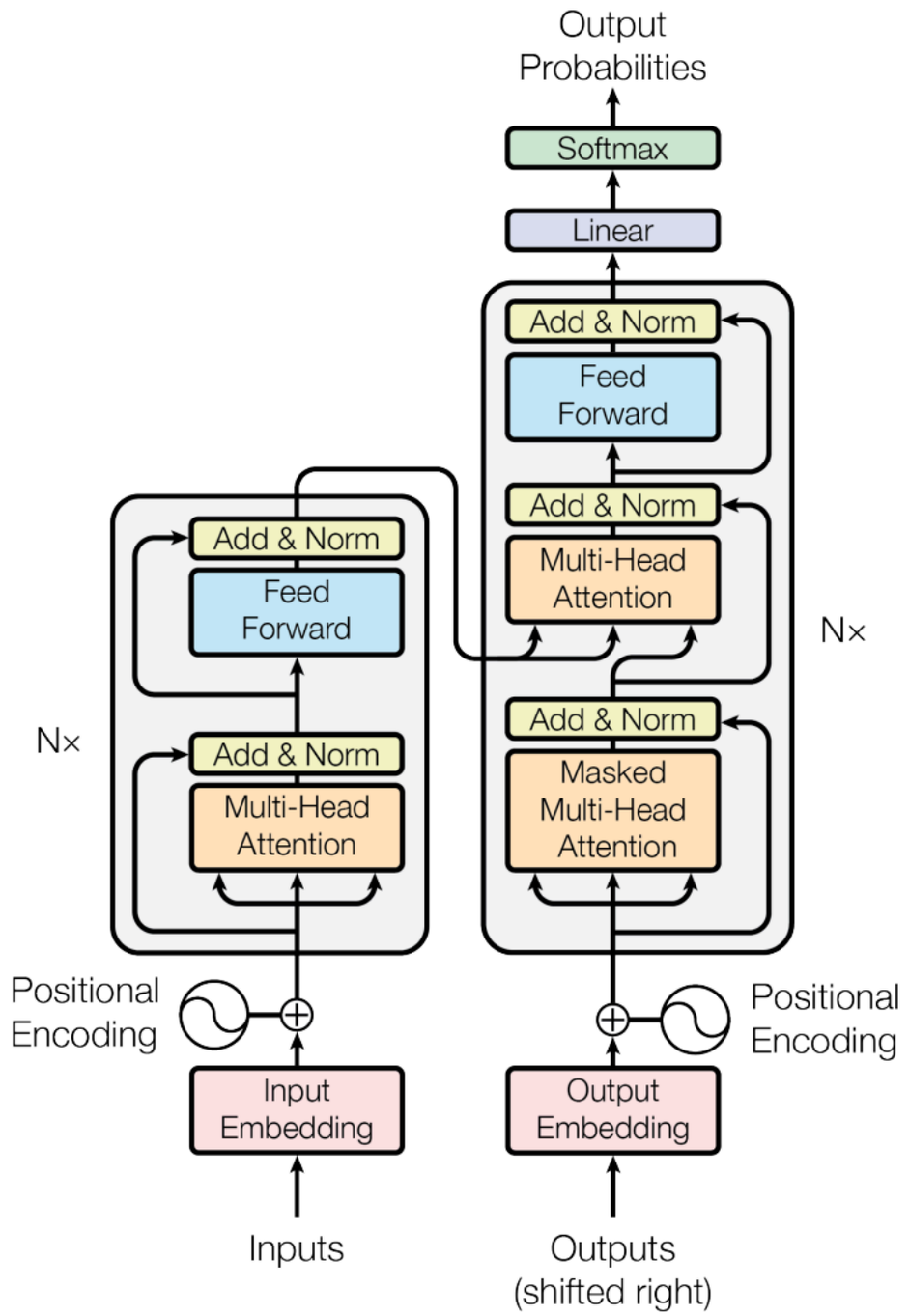
- Big guns ?
  - Multi-Head Self-Attention
  - Positional Encoding



Anatomy of a Transformer

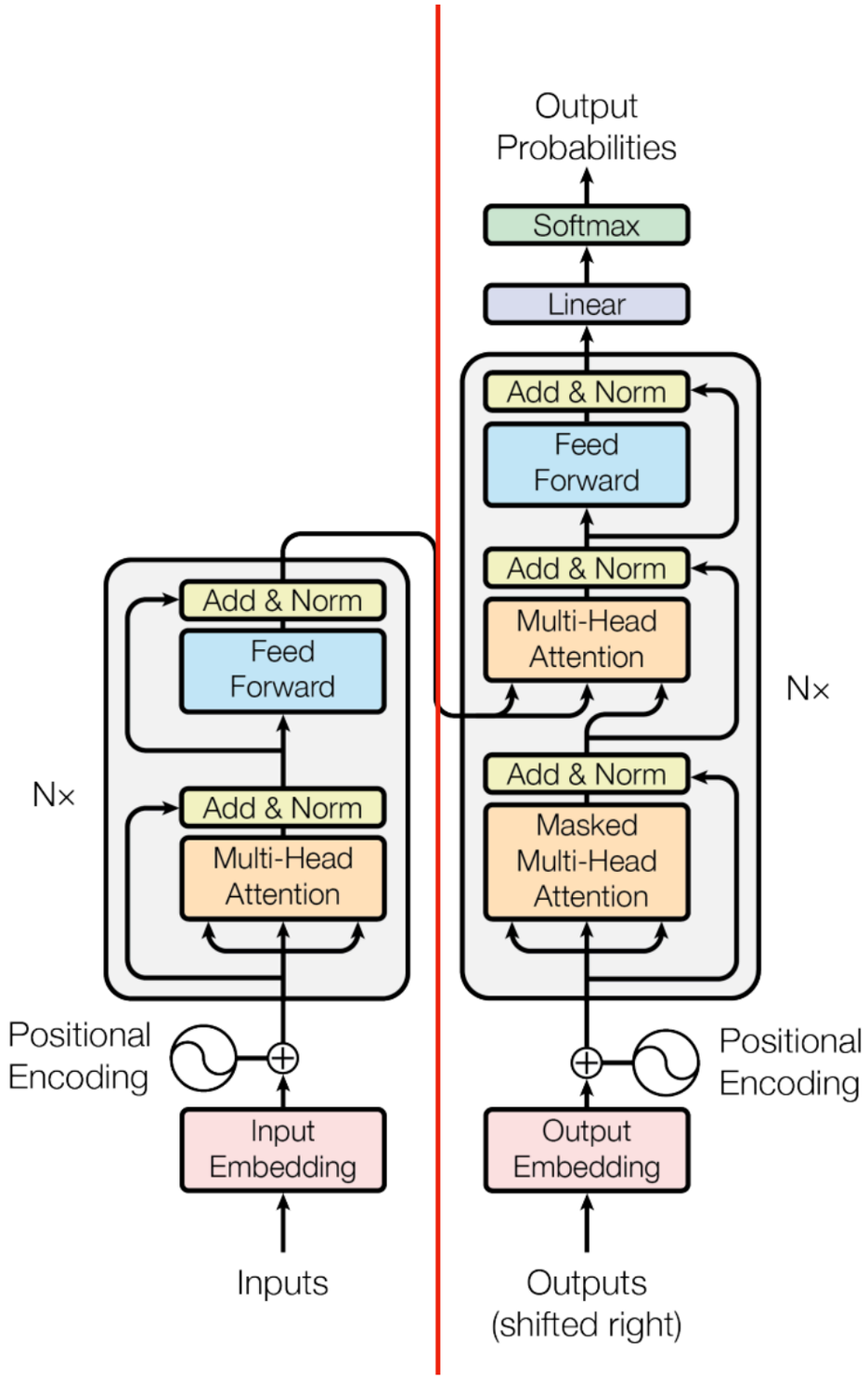
- Huge models and requires a lot of input data it is not new, we already had models bigger than this!

- Big guns ?
  - Multi-Head Self-Attention
  - Positional Encoding
- And Its amazing!



Anatomy of a Transformer

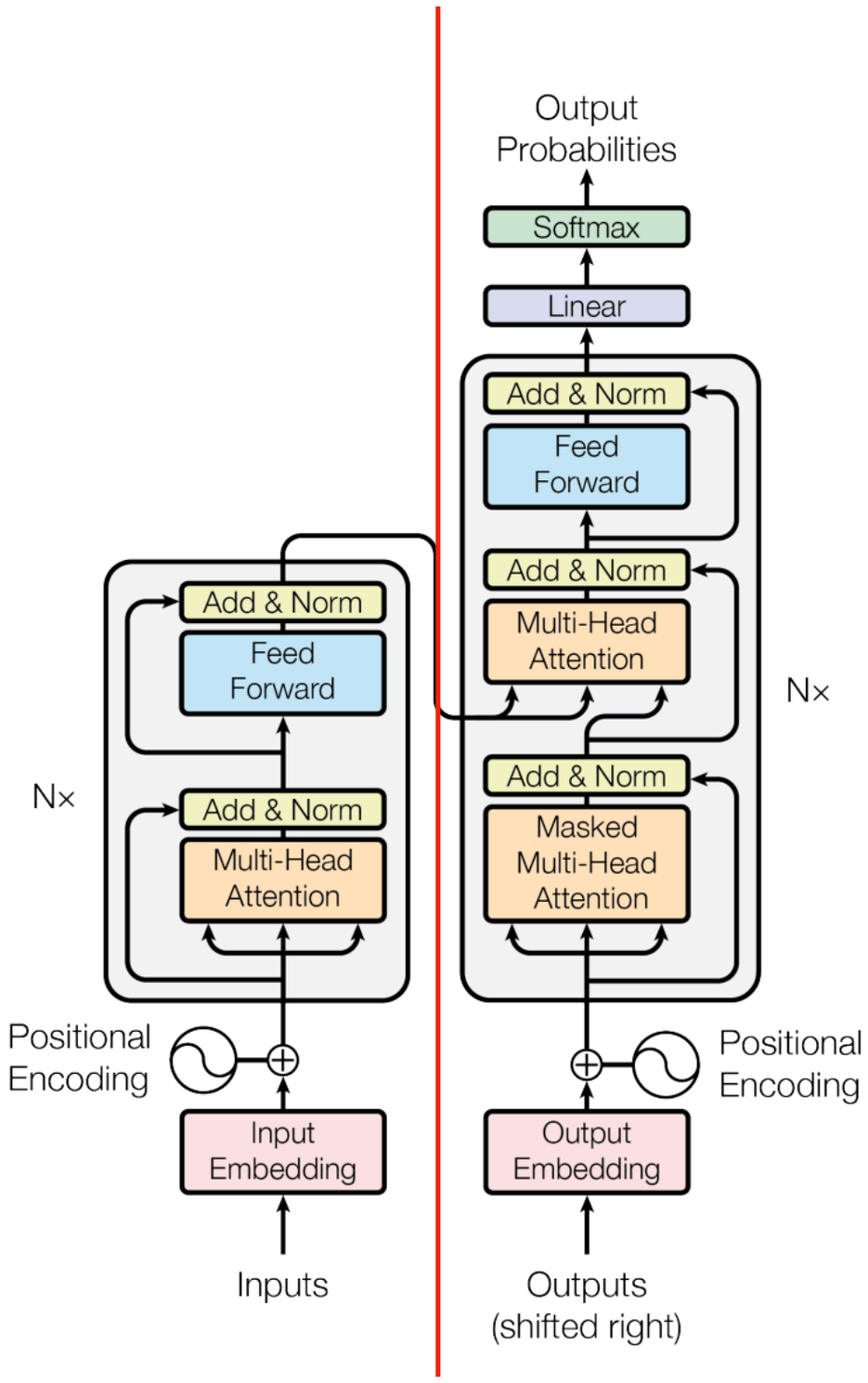
Encoder



Decoder

Anatomy of a Transformer

$N\times$ Encoders



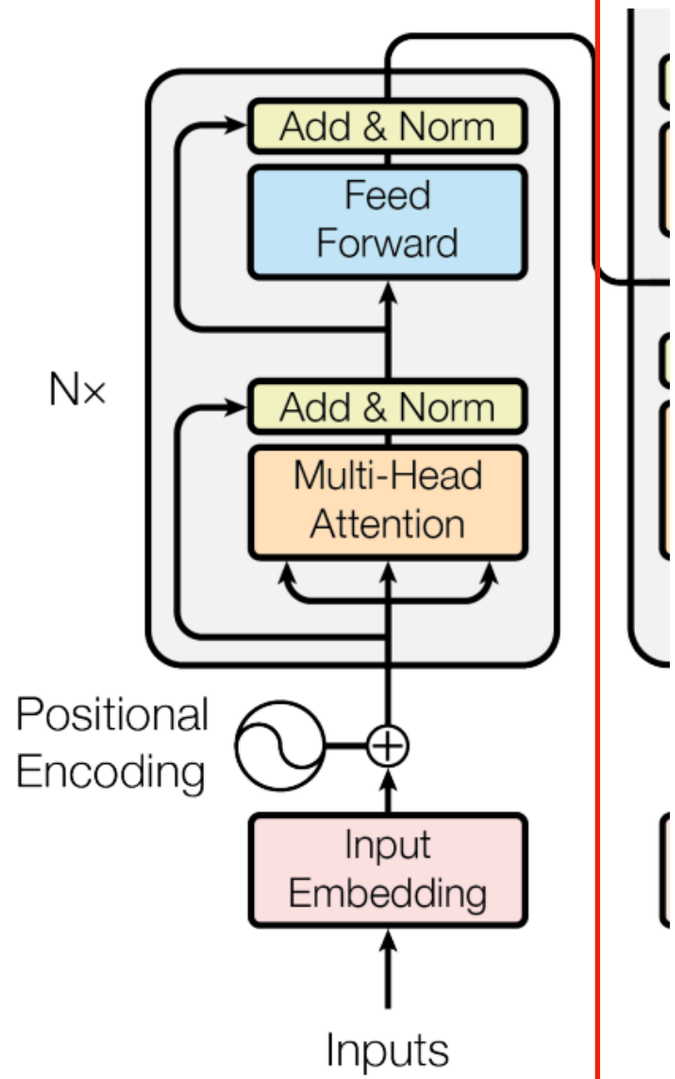
$N\times$ Decoders

Anatomy of a Transformer

- Each encoder has multi-head attention layers and feed forward layers

- The input is fed all at once!

- How? What is a multi-head attention layer?



Attention (RNNs)

- In the Encoder Decoder model “state” phase of mapping, instead of the direct mapping sequential (as we have done before!)

  - $c = m(h_1, \ldots, h_t)$

- We do a mapping that looks at the entire encoders sequence and checks relevance with the decoder states:

$$c_i = \sum_j \alpha_{i,j}[h_j; \overline{h_j}]^T \, , \alpha_{i,j} = \frac{exp(s_{i,j})}{\sum_k exp(s_{i,j})}$$

Attention (RNNs)

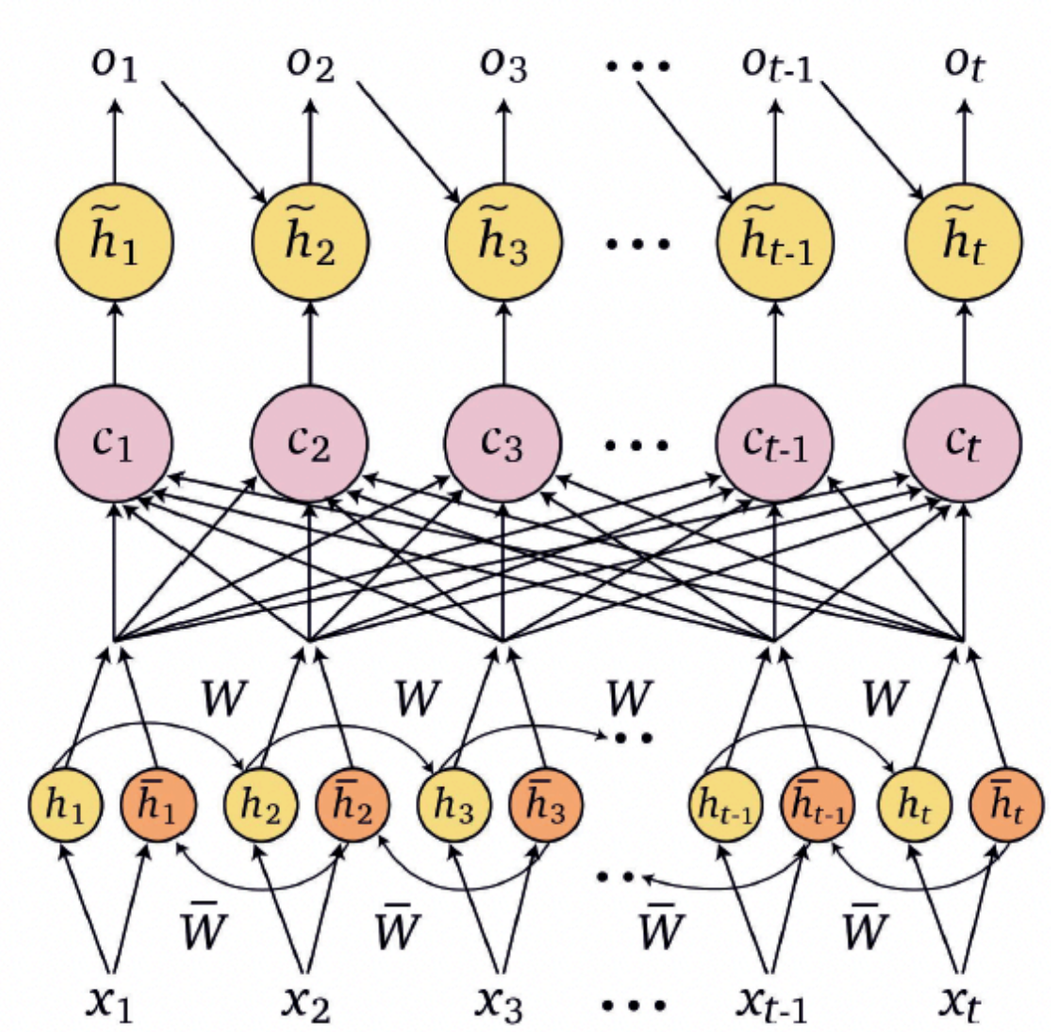
$c_i = \sum_j \alpha_{i,j}[h_j; \overline{h_j}]^T$ does the mapping

$$\alpha_{i,j} = \frac{exp(s_{i,j})}{\sum_k exp(s_{i,j})}$$

(basically applies Softmax activation)

$s_{i,j}$ function of encoder $[h_j; \overline{h_j}]^T$ and decoder $\tilde{h}_t$

This mechanism computes the relevance of each encoder h_t to a specific decoder state $\tilde{h}_t$ during the decoding



Self-Attention (2017)

- The idea follows the principle of the attention in RNNs but it takes into account all words at once in parallel.

- $x_i \in \mathbb{R}^d$, $X = [x_1, \ldots, x_n]^T \in \mathbb{R}^{n \times d}$, $n \times d$ matrix

- The self-attention computes self attention representation with every word with itself.

Self-Attention (2017)

- For each $x_i \in \mathbb{R}^d$ we compute
- query $q_i \in \mathbb{R}^d$
- key $k_i \in \mathbb{R}^d$
- value $v_i \in \mathbb{R}^d$
- And by early projecting X we have the following matrices
- $Q = XW^Q, K = XW^K, V = XW^V$

Self-Attention (2017)

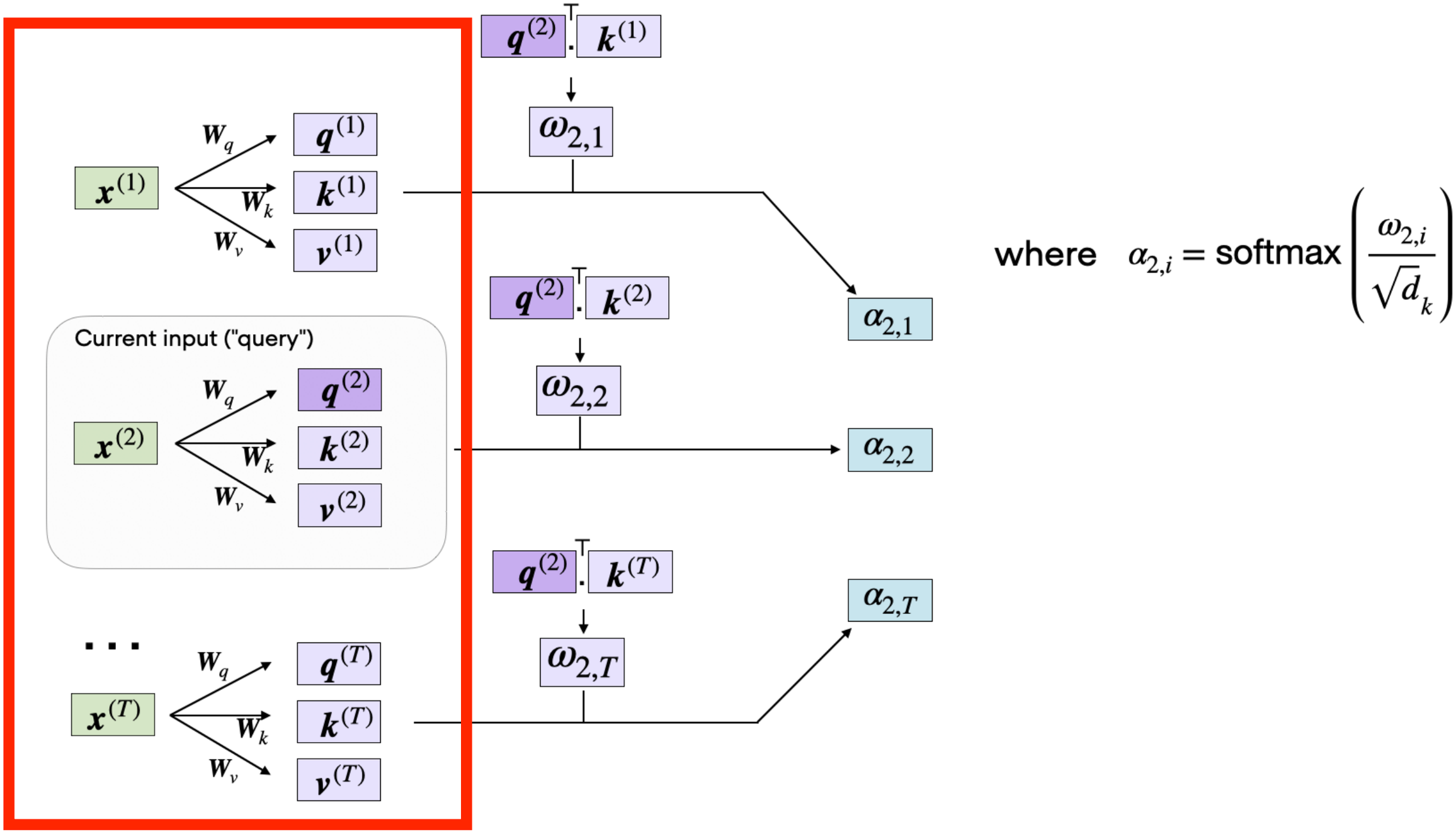
- And by early projecting X we have the following matrices

- $Q = XW^Q, K = XW^K, V = XW^V$

- We then compute attention scores A_i with q_i and k_i vectors using the dot product. Similar queries and keys yield high dot products.

$$A_i(q, K, V) = \sum_{i=1}^{n} \frac{\exp(qk_i)}{\sum_j \exp(qk_j)} v_i \qquad A(X) = A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

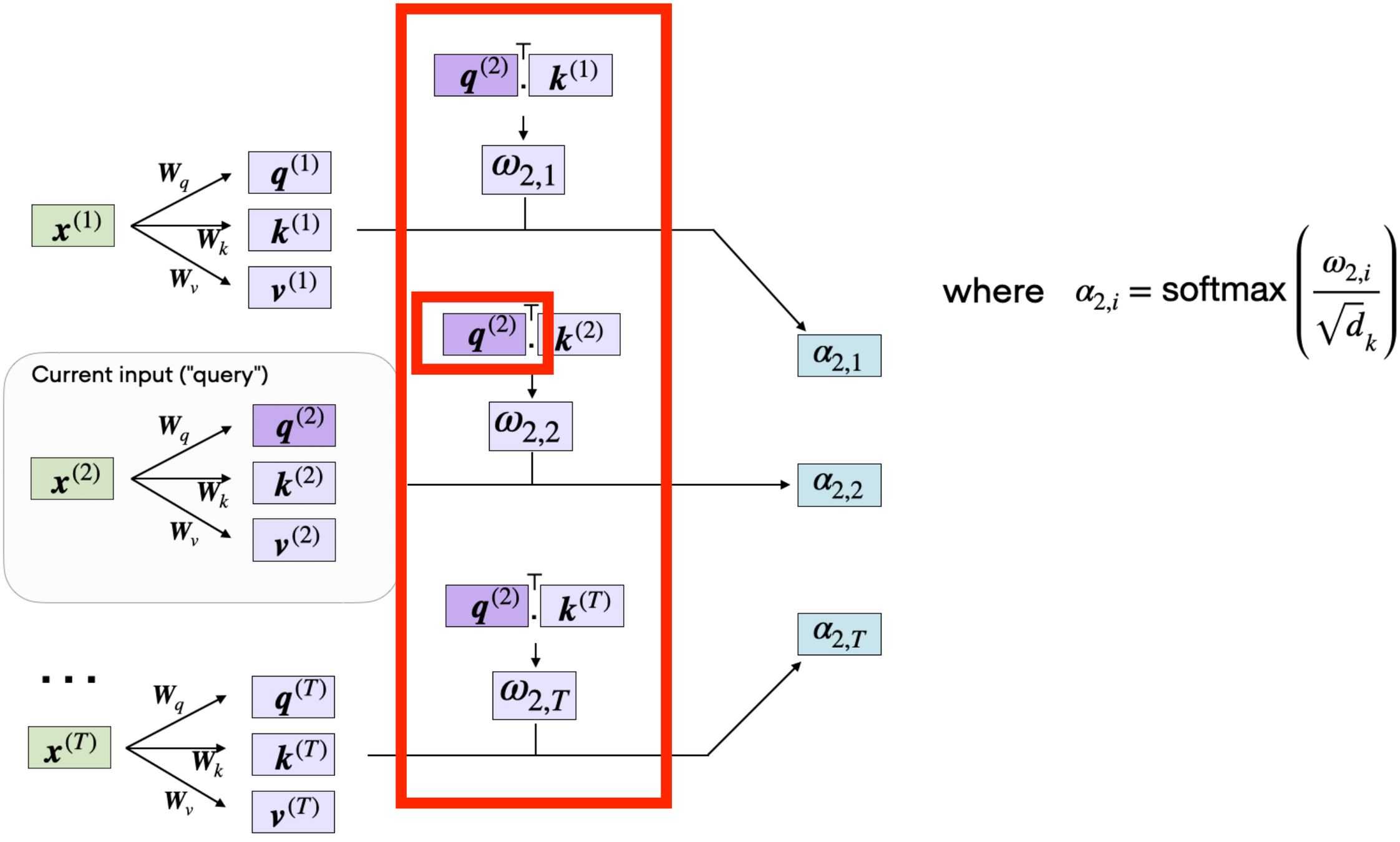
Self-Attention (2017)



where $\alpha_{2,i} = \text{softmax}\left(\frac{\omega_{2,i}}{\sqrt{d_k}}\right)$

https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html

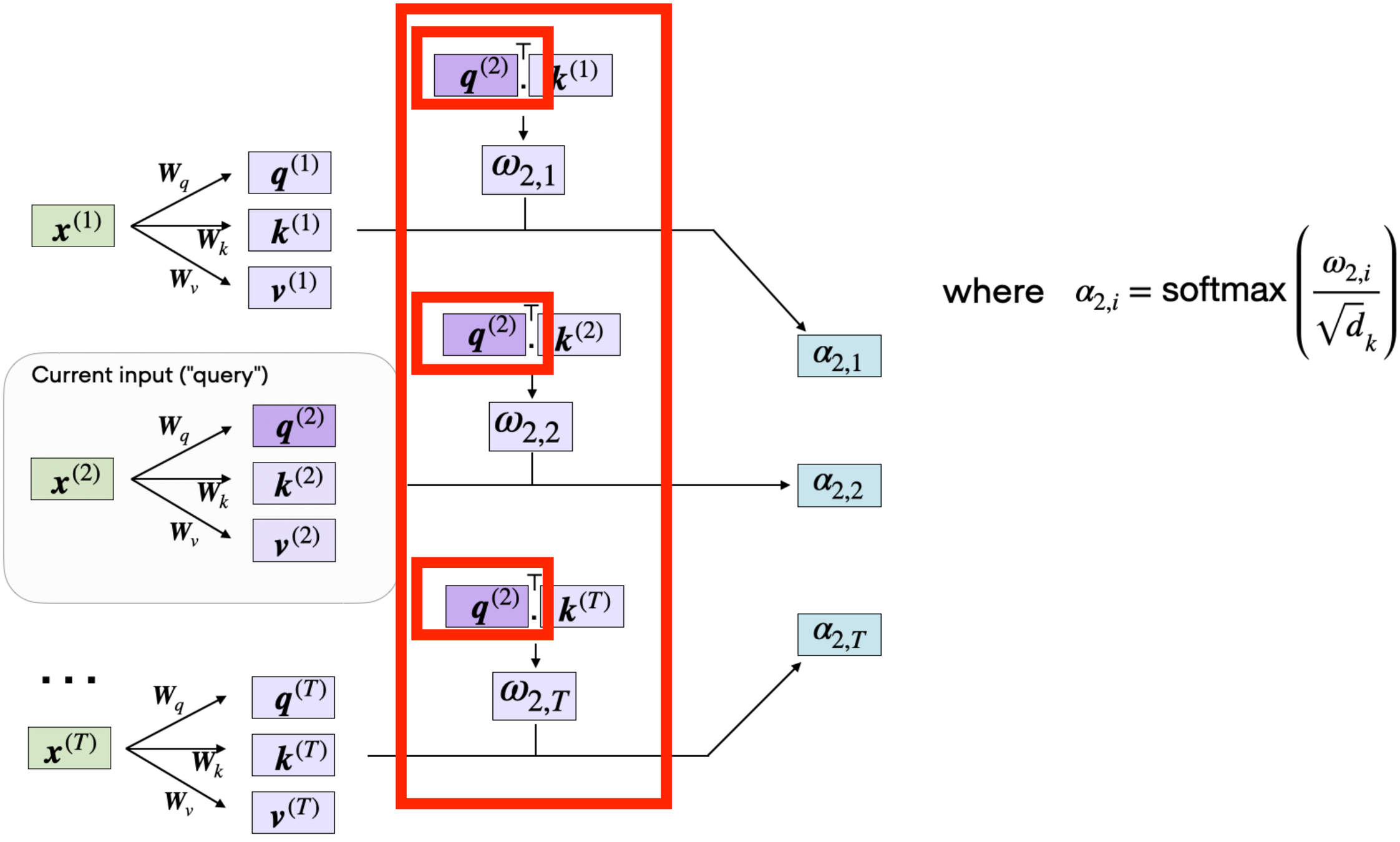
Self-Attention (2017)



where $\alpha_{2,i} = \text{softmax}\left(\frac{\omega_{2,i}}{\sqrt{d_k}}\right)$

https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html

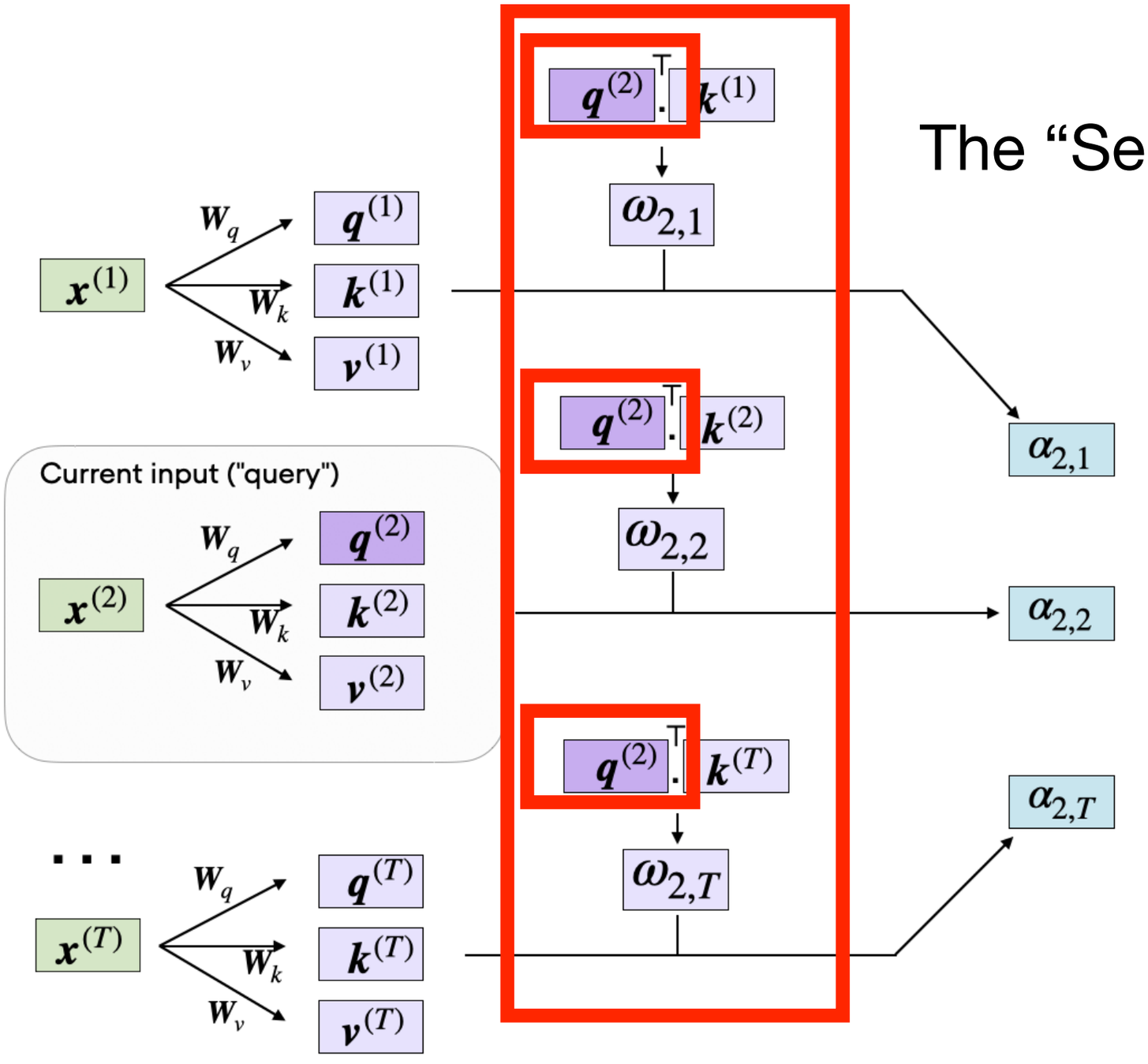
Self-Attention (2017)



where $\alpha_{2,i} = \text{softmax}\left(\frac{\omega_{2,i}}{\sqrt{d_k}}\right)$

https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html

Self-Attention (2017)

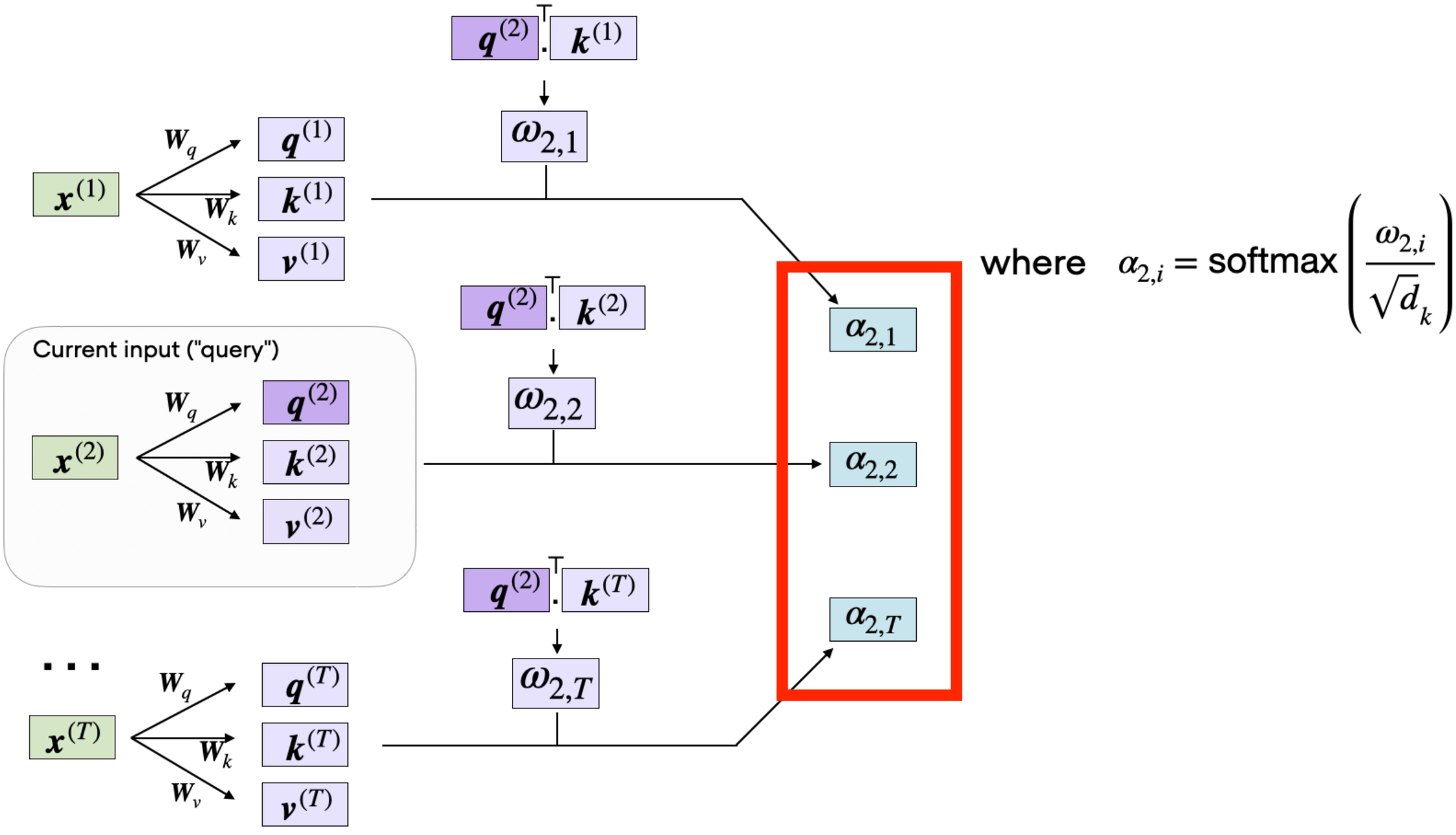


The “Self” in Self-Attention

where $\alpha_{2,i} = \text{softmax}\left(\frac{\omega_{2,i}}{\sqrt{d_k}}\right)$

https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html

Self-Attention (2017)



https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html

Self-Attention (2017)

Simple example with 2 tokens

Q checks for info

$$Q = \begin{pmatrix} q_A \\ q_B \end{pmatrix}$$

K provides the info

$$K = \begin{pmatrix} k_A \\ k_B \end{pmatrix}$$

$$Q \cdot K^T = \begin{pmatrix} q_A \cdot k_A & q_A \cdot k_B \\ q_B \cdot k_A & q_B \cdot k_B \end{pmatrix}$$

https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html

Self-Attention (2017)

Simple example with 2 tokens in 3d embedding space!

Q checks for info

K provides the info

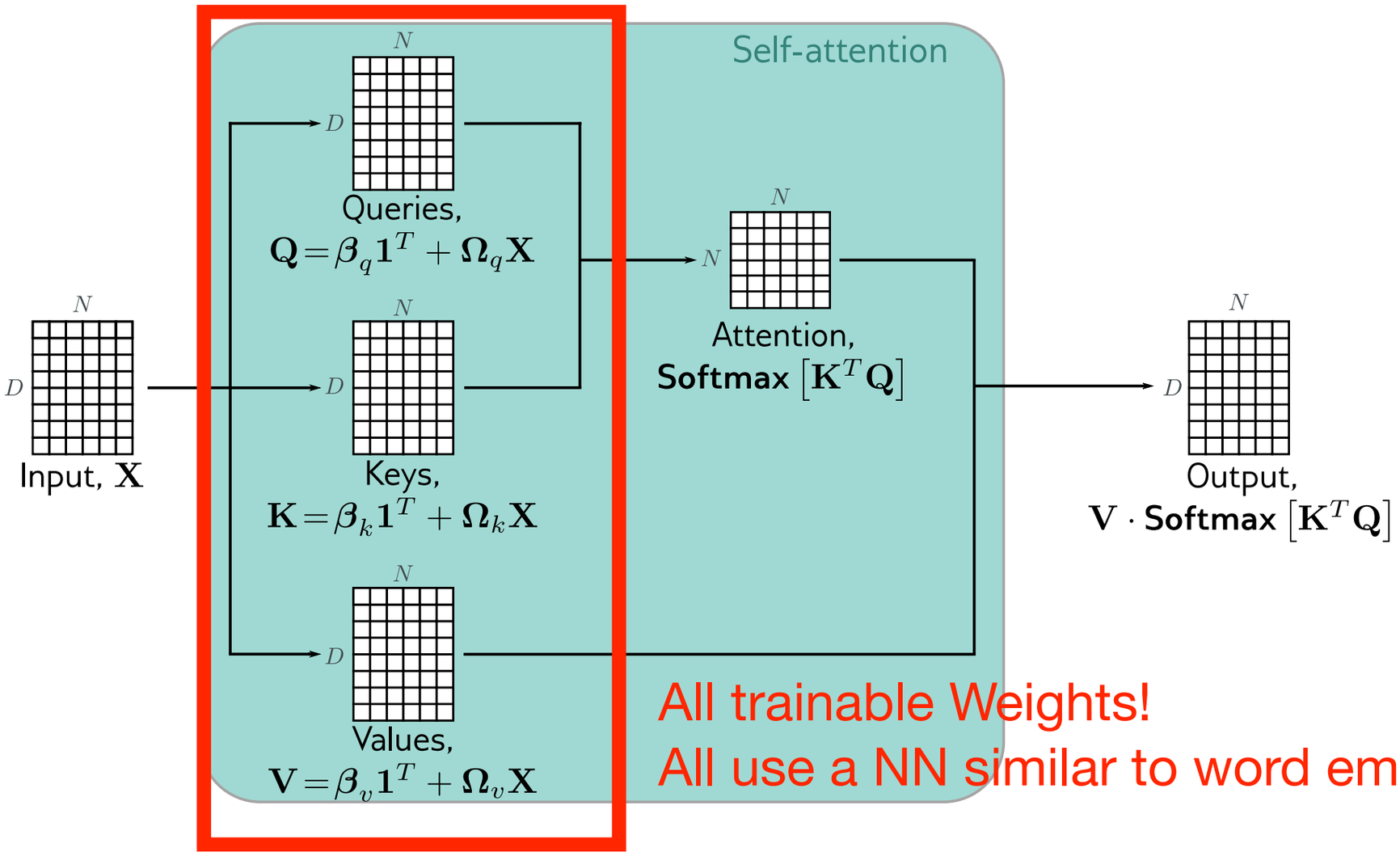
$$Q = \begin{pmatrix} q_{11} & q_{12} & q_{13} \\ q_{21} & q_{22} & q_{23} \end{pmatrix} \qquad K = \begin{pmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \end{pmatrix}$$

$$Q \cdot K^T = \begin{pmatrix} q_{11}k_{11} + q_{12}k_{12} + q_{13}k_{13} & q_{11}k_{21} + q_{12}k_{22} + q_{13}k_{23} \\ q_{21}k_{11} + q_{22}k_{12} + q_{23}k_{13} & q_{21}k_{21} + q_{22}k_{22} + q_{23}k_{23} \end{pmatrix}$$

Attention Scores!

https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html

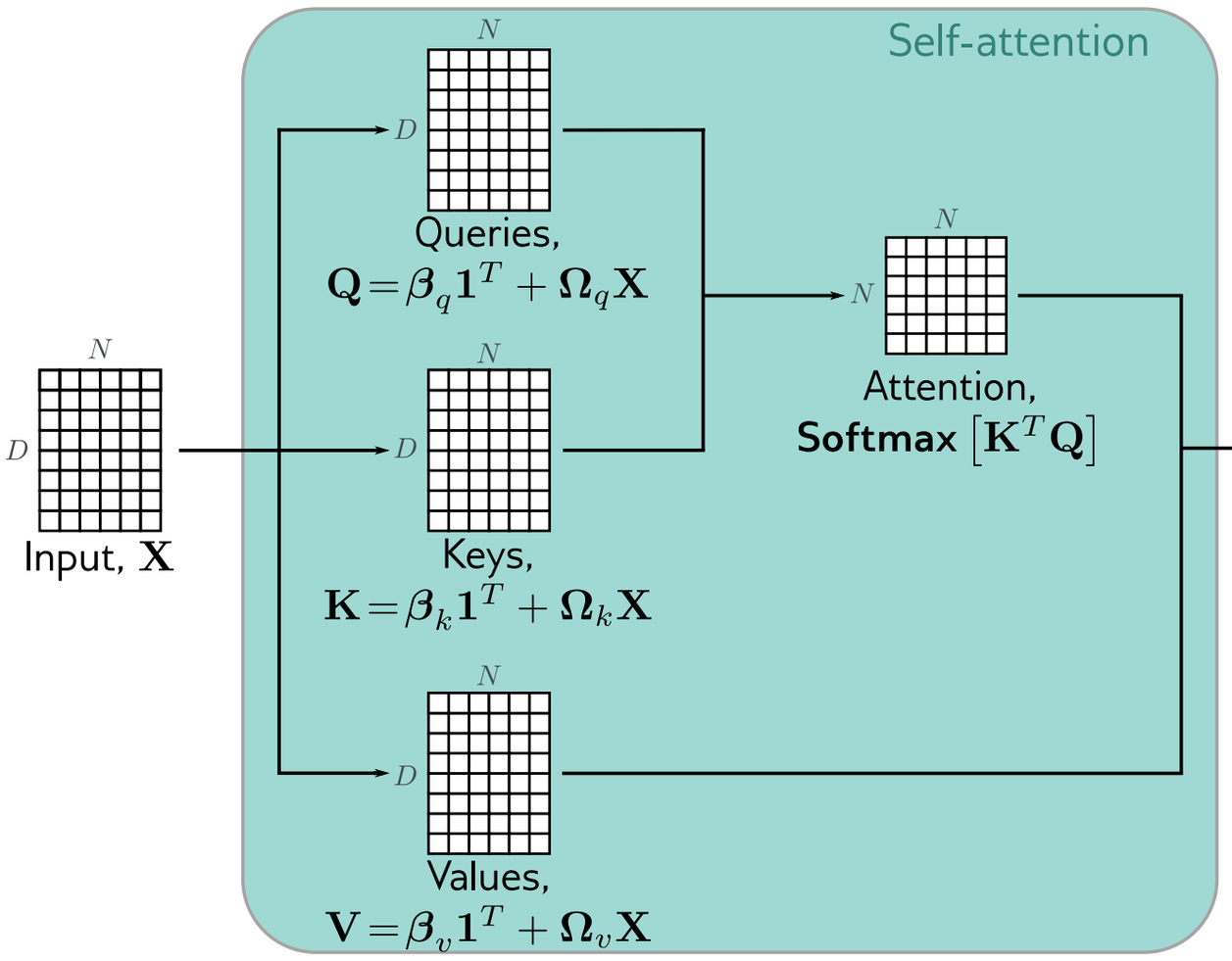
Self-Attention (2017)



All trainable Weights!

All use a NN similar to word embeddings

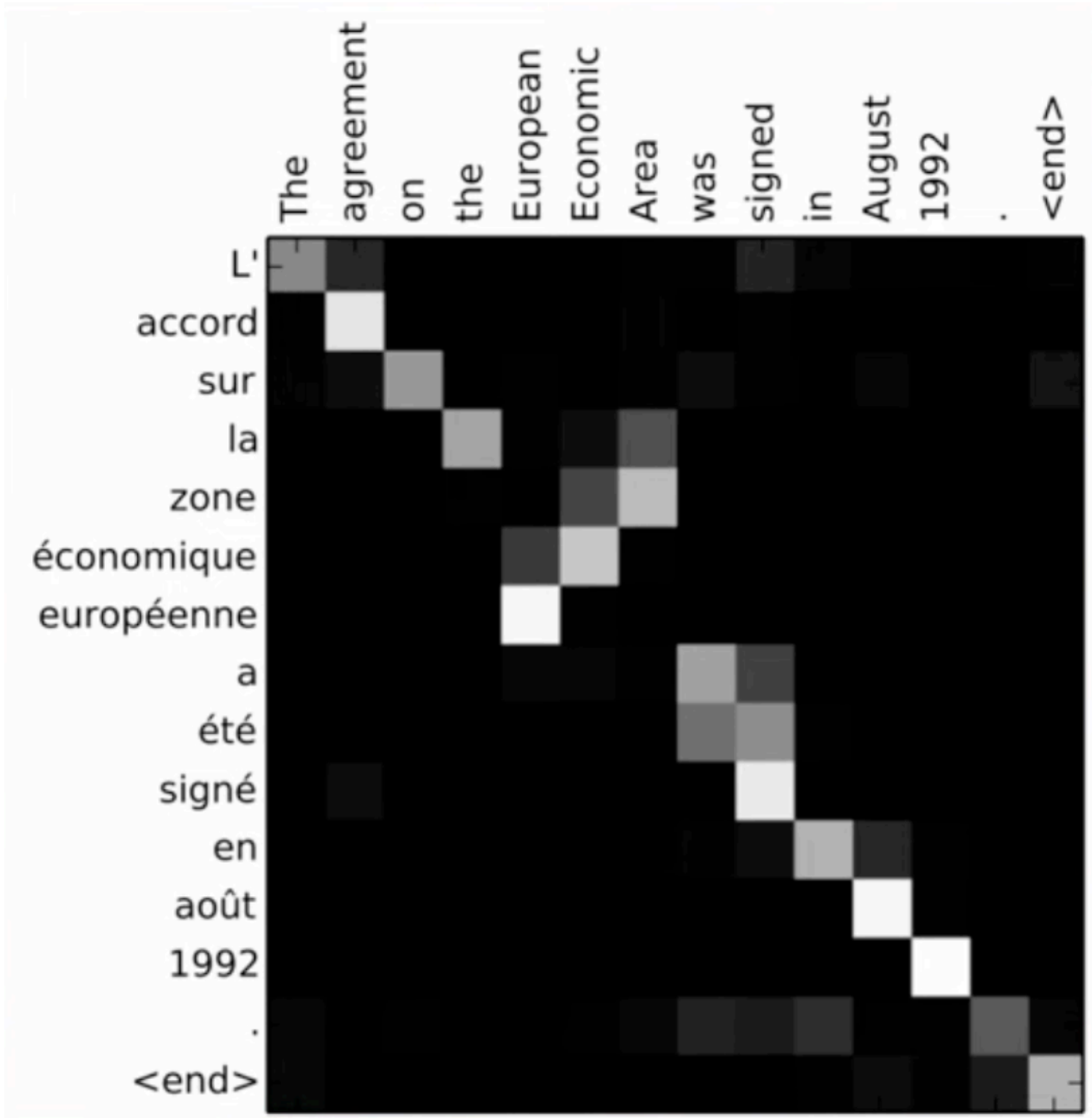
Self-Attention (2017)



Final Representation

It forms the Contextual Embedding!

Self-Attention

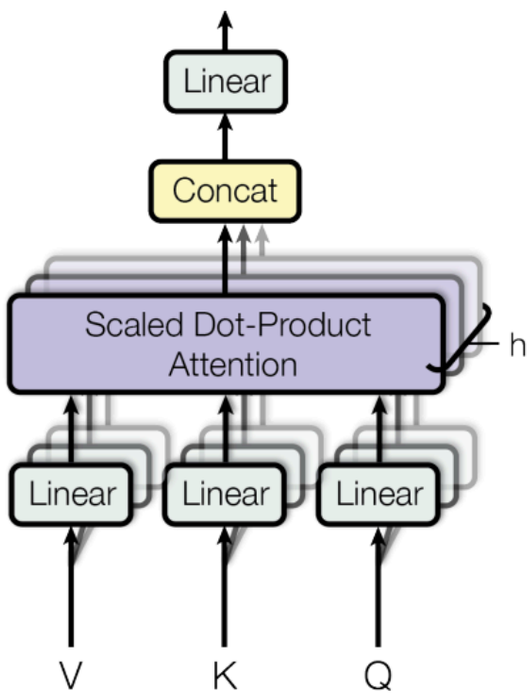


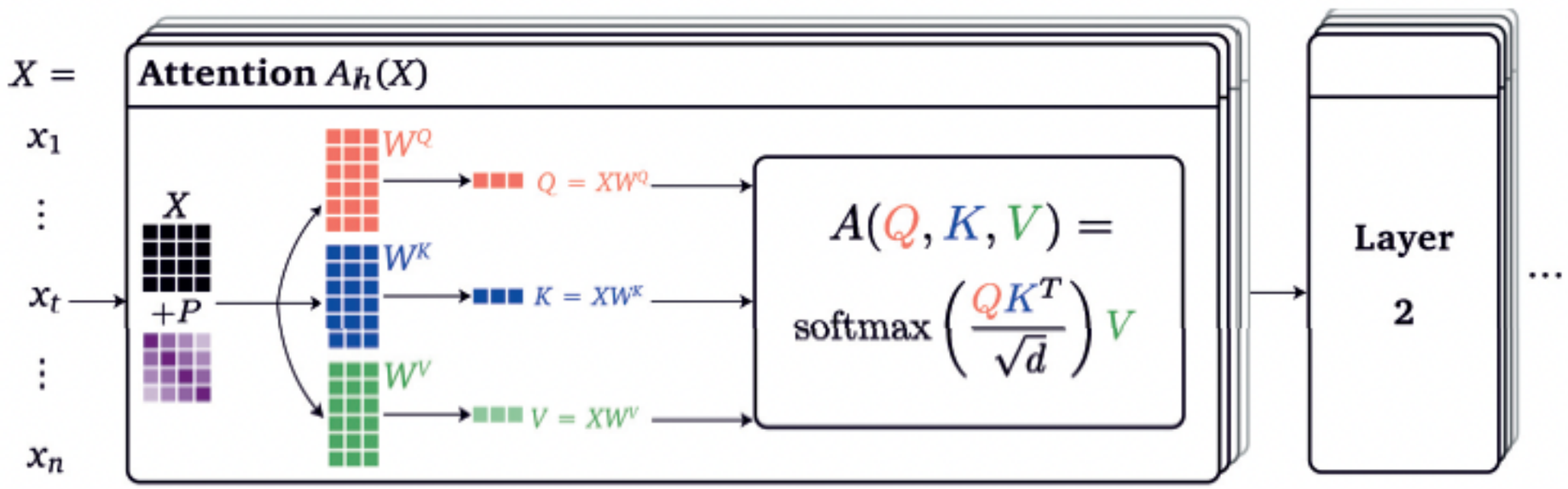
Multi-head Attention

- With multi-head Attention we just perform self-attention multiple times!

$$A_h(X) = A_h(Q, K, V) = A_h(W_h^Q Q, W_h^K K, W_h^V V) = \text{softmax}\left(\frac{Q_h K_h^T}{\sqrt{d}}\right) V_h$$

Different layers with different set of weights



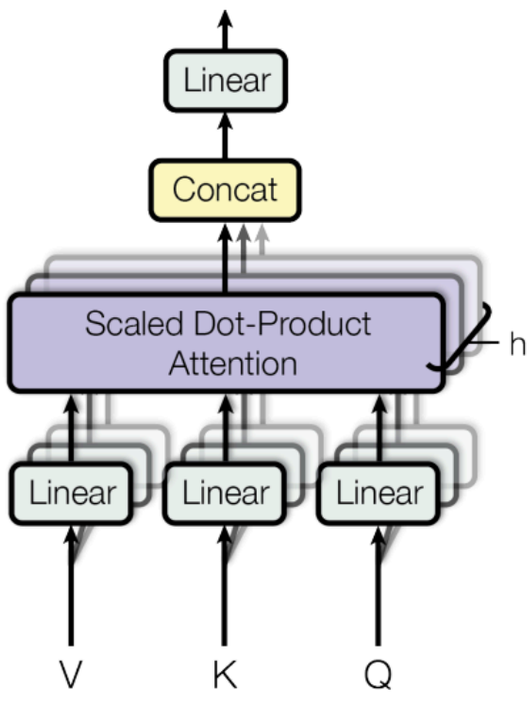


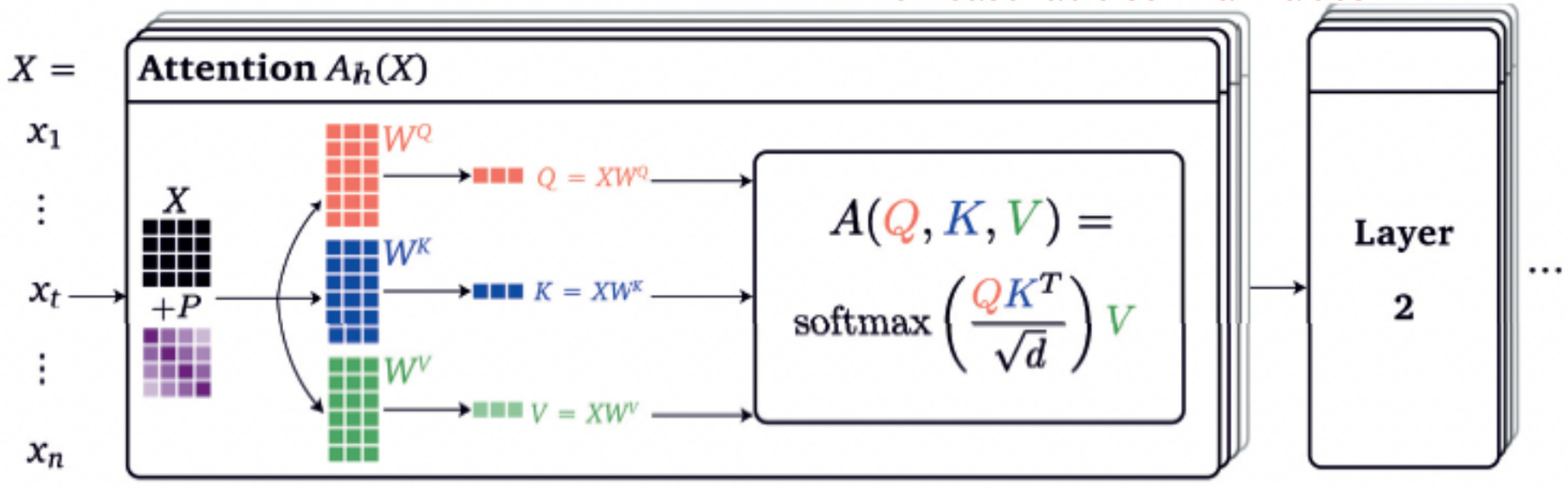
Multi-head Attention

- With multi-head Attention we just perform self-attention multiple times!

$$A_h(X) = A_h(Q, K, V) = A_h(W_h^Q Q, W_h^K K, W_h^V V) = \text{softmax}\left(\frac{Q_h K_h^T}{\sqrt{d}}\right) V_h$$

it is normalised to the dimension of K vector for reasonable softmax values!

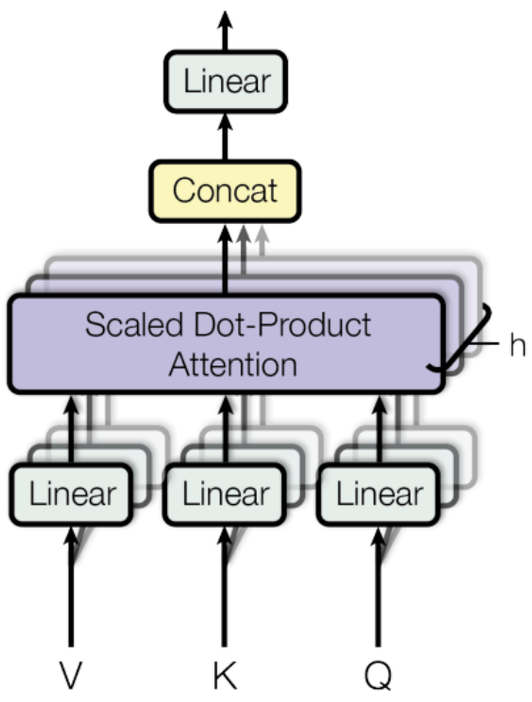


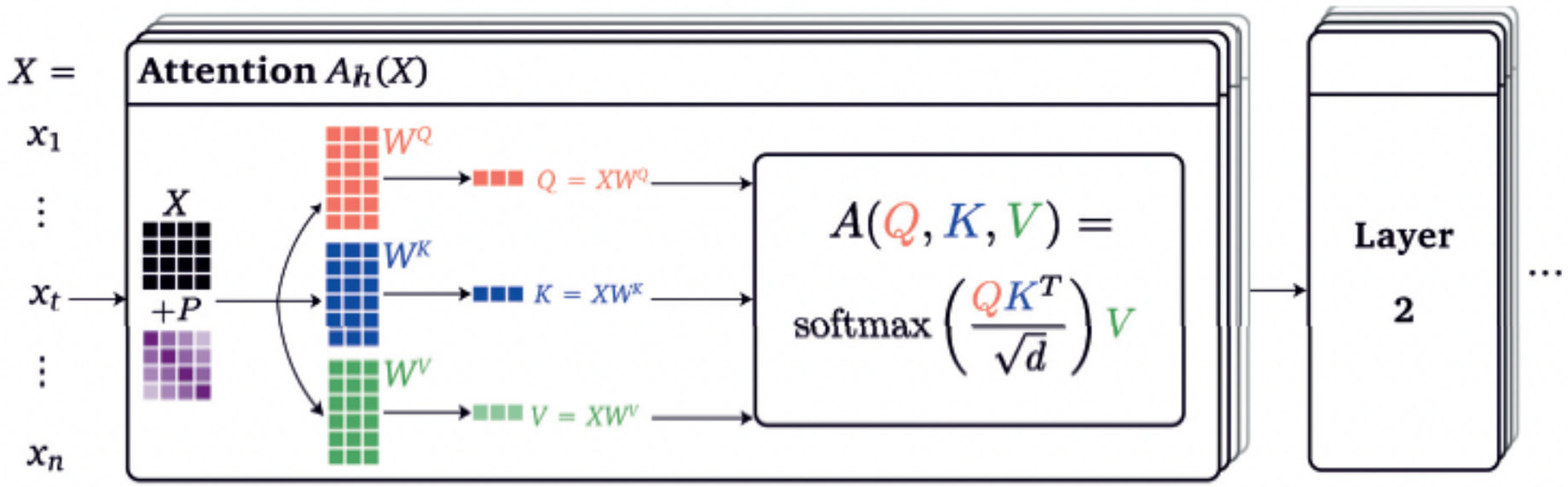


Multi-head Attention

- With multi-head Attention we just perform self-attention multiple times!

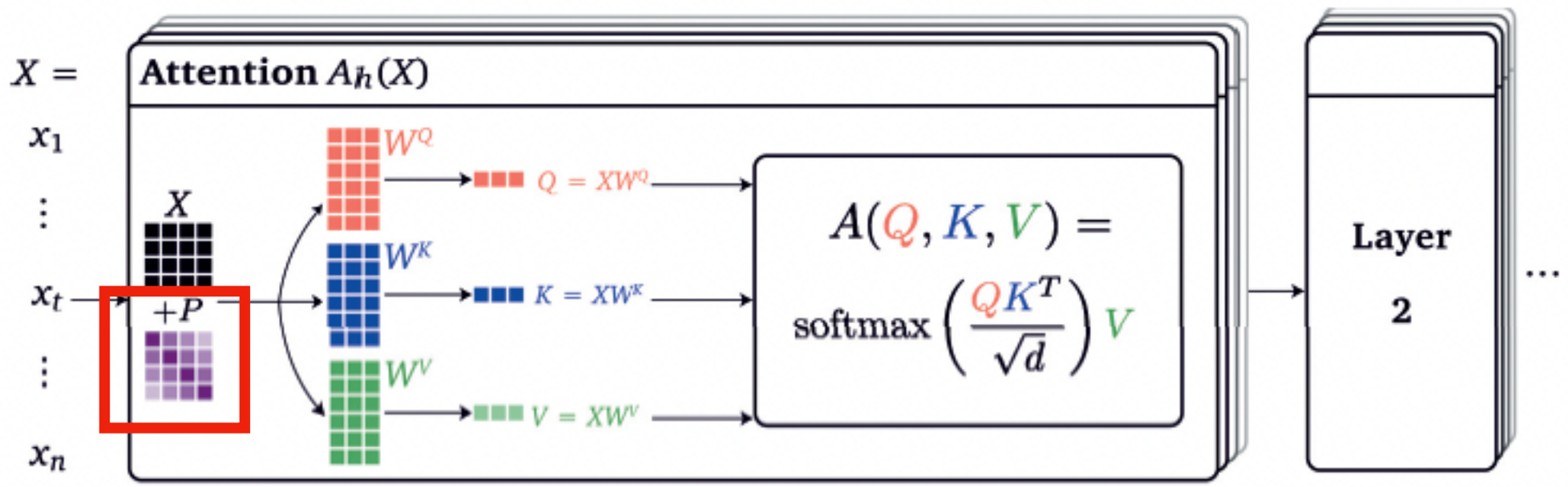
$$A_h(X) = A_h(Q, K, V) = A_h(W_h^Q Q, W_h^K K, W_h^V V) = \text{softmax}\left(\frac{Q_h K_h^T}{\sqrt{d}}\right) V_h$$





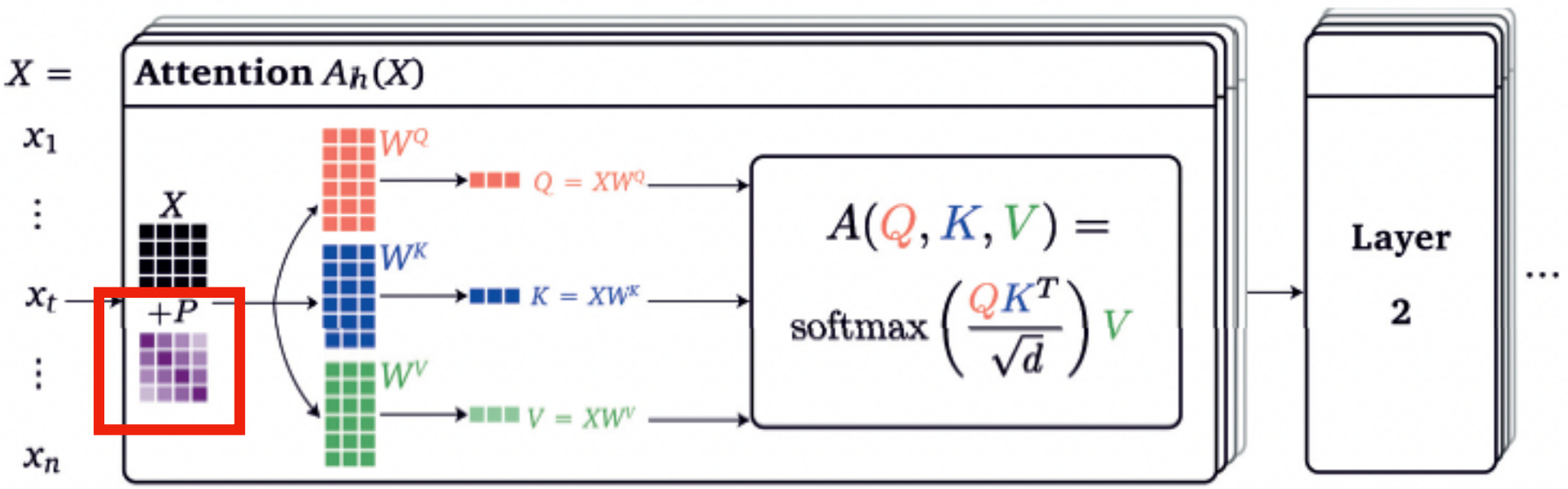
Multi-head Attention

- Whats that P?



Positional Encoding!

- Whats that P?

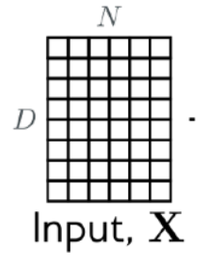


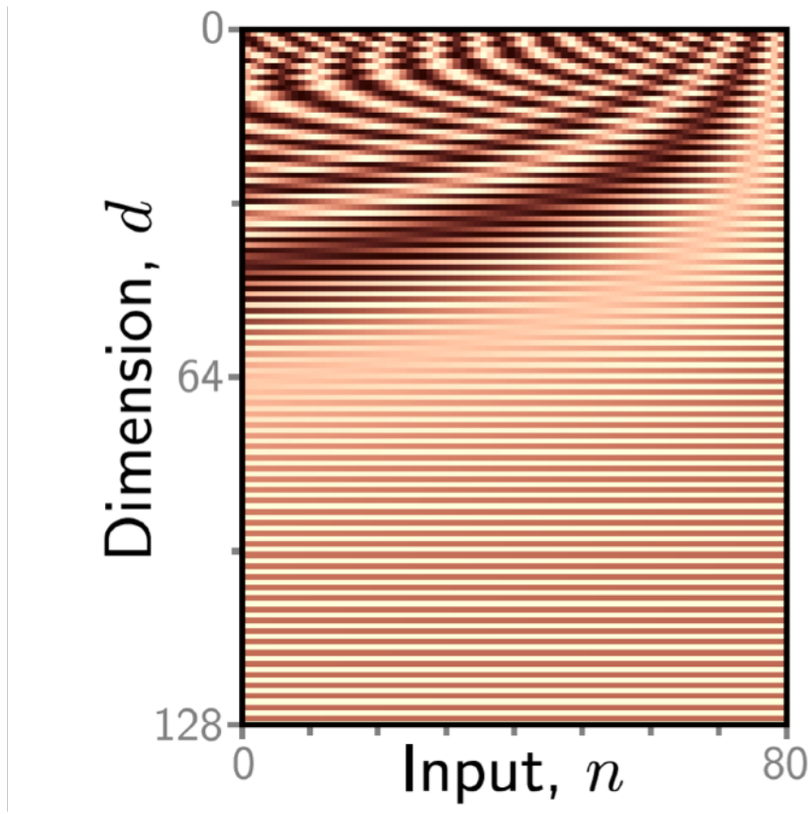
Positional Encoding

- Problem with self-attention is that does not take into account order!
    - Lunch eats Man
    - Man eats Lunch
- Order matters!

Positional Encoding

- Assign cosine and sine positional values at different input lengths
  - It ensures they are unique!
- Ensures that tokens on different positions of the input will have different embeddings!





$$P_{pos,2i} = \sin\left(\frac{pos}{10,000^{\frac{2i}{d}}}\right), \quad P_{\text{pos},2i+1} = \cos\left(\frac{pos}{10,000^{\frac{2i}{d}}}\right)$$

Positional Encoding

- Simple example with input tokens with 3d embeddings

$$PE(pos,0) = \sin\left(\frac{pos}{10000^{0/3}}\right) = \sin(pos)$$

$$PE(pos,1) = \cos\left(\frac{pos}{10000^{0/3}}\right) = \cos(pos)$$

Typically sin for even indices and cos for odd indices

$$PE(pos,2) = \sin\left(\frac{pos}{10000^{2/3}}\right)$$

$$pos = 1: \quad PE(1) = \left[\sin(1),\ \cos(1),\ \sin\left(\frac{1}{10000^{2/3}}\right)\right]$$

$$pos = 2: \quad PE(2) = \left[\sin(2),\ \cos(2),\ \sin\left(\frac{2}{10000^{2/3}}\right)\right]$$

Positional Encoding

- Simple example with input tokens with 3d embeddings

$$PE(pos,0) = \sin\left(\frac{pos}{10000^{0/3}}\right) = \sin(pos)$$

$$PE(pos,1) = \cos\left(\frac{pos}{10000^{0/3}}\right) = \cos(pos)$$

$$PE(pos,2) = \sin\left(\frac{pos}{10000^{2/3}}\right)$$

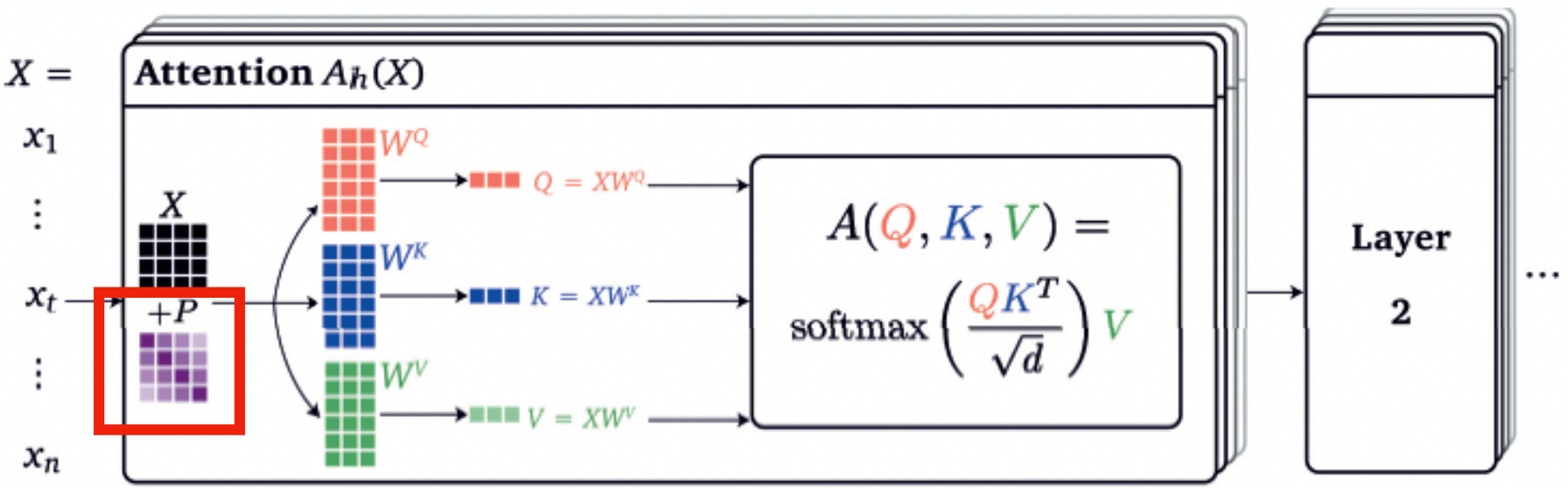
$$pos = 1: \quad PE(1) = \left[\sin(1),\ \cos(1),\ \sin\left(\frac{1}{10000^{2/3}}\right)\right]$$

$$pos = 2: \quad PE(2) = \left[\sin(2),\ \cos(2),\ \sin\left(\frac{2}{10000^{2/3}}\right)\right]$$

Positional Encodings are added to the Tokens Embeddings

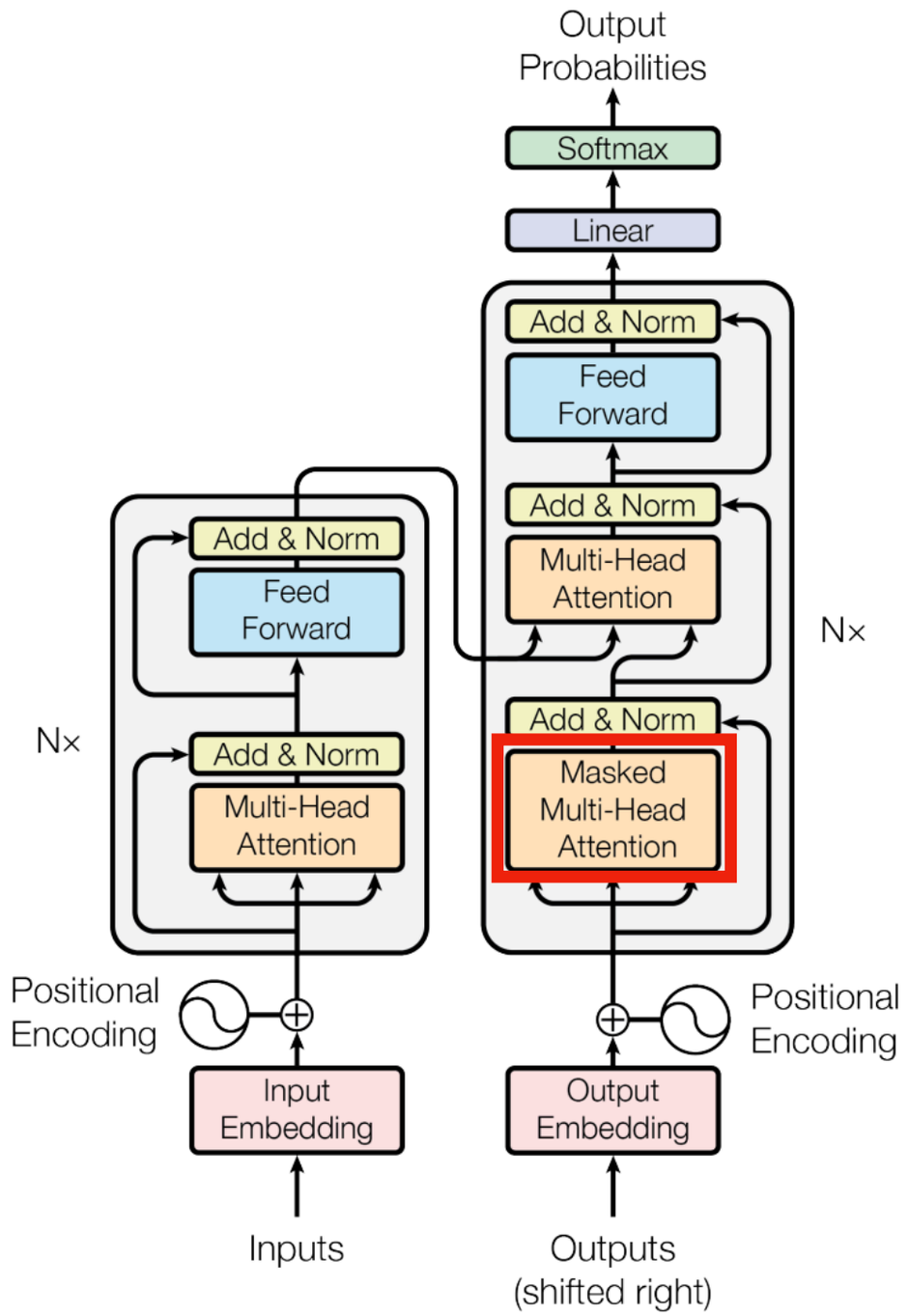
Positional Encoding!

- Whats that P? Solved!



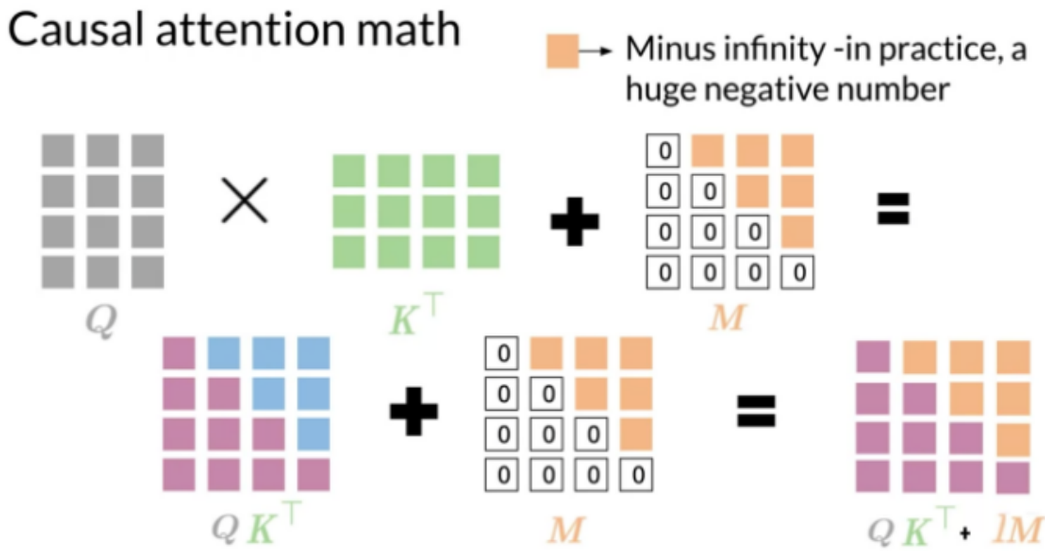
Anatomy of a Transformer

- On the decoder side the only difference is that masked attention layers (or causal self attention) only the tokens before a input token are compared with that token in the sequence of tokens



Anatomy of a Transformer

- On the decoder side the only difference is that masked attention layers (or causal self attention) only the tokens before a input token are compared with that token in the sequence of tokens

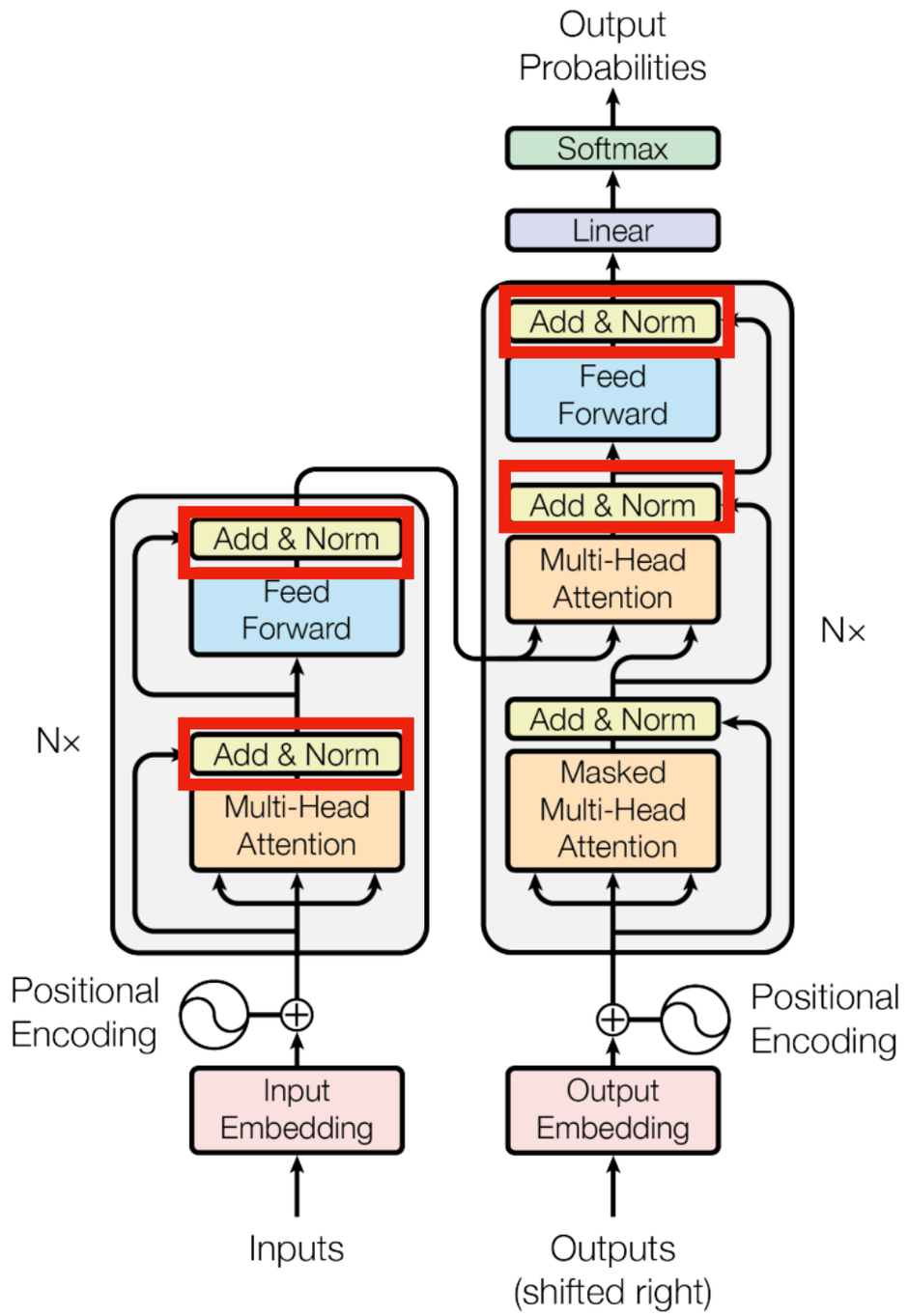


- We cannot check the future when decoding!
  - At inference time we will not know!

Anatomy of a Transformer

- On the decoder side the only difference is that masked attention layers (or causal self attention) only the tokens before a input token are compared with that token in the sequence of tokens

- We have multiple Batch Normalisation layers (CNNs class) that are added to skip connections (ResNets class) between multihead attention layers.



Did we made it?

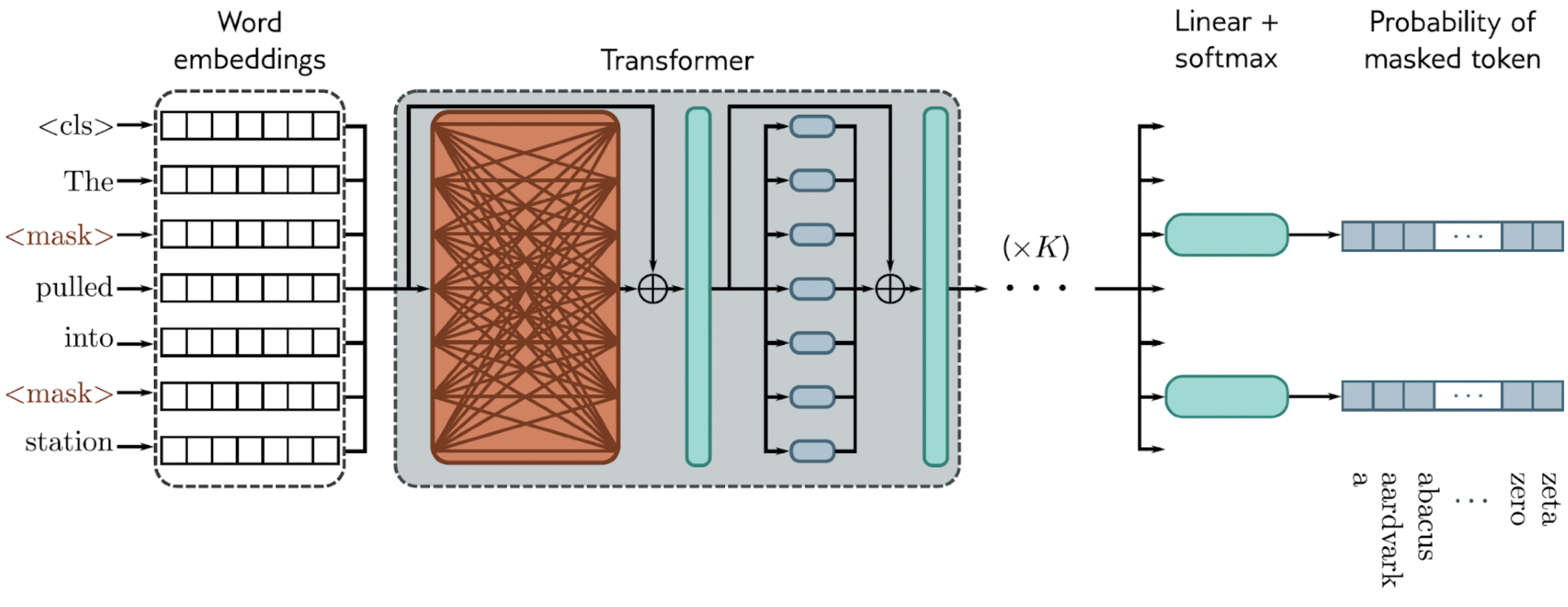
Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Examples

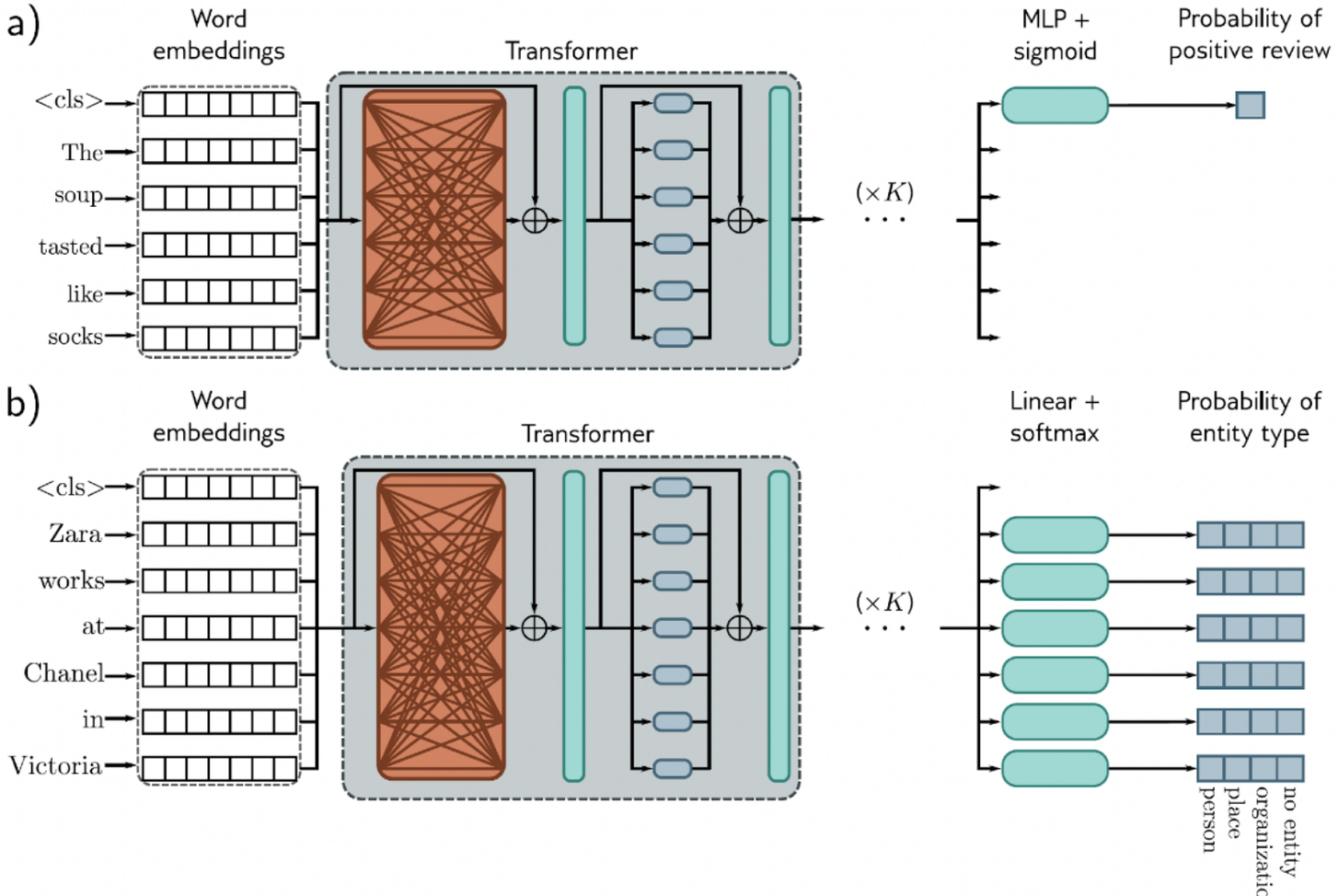
BERT

- Bidirectional Encoder Representations from Transformers (BERT) - Learns directional representations from inputs
    - Pre-trained for predicting random words or arrangements of sentences/group of words.



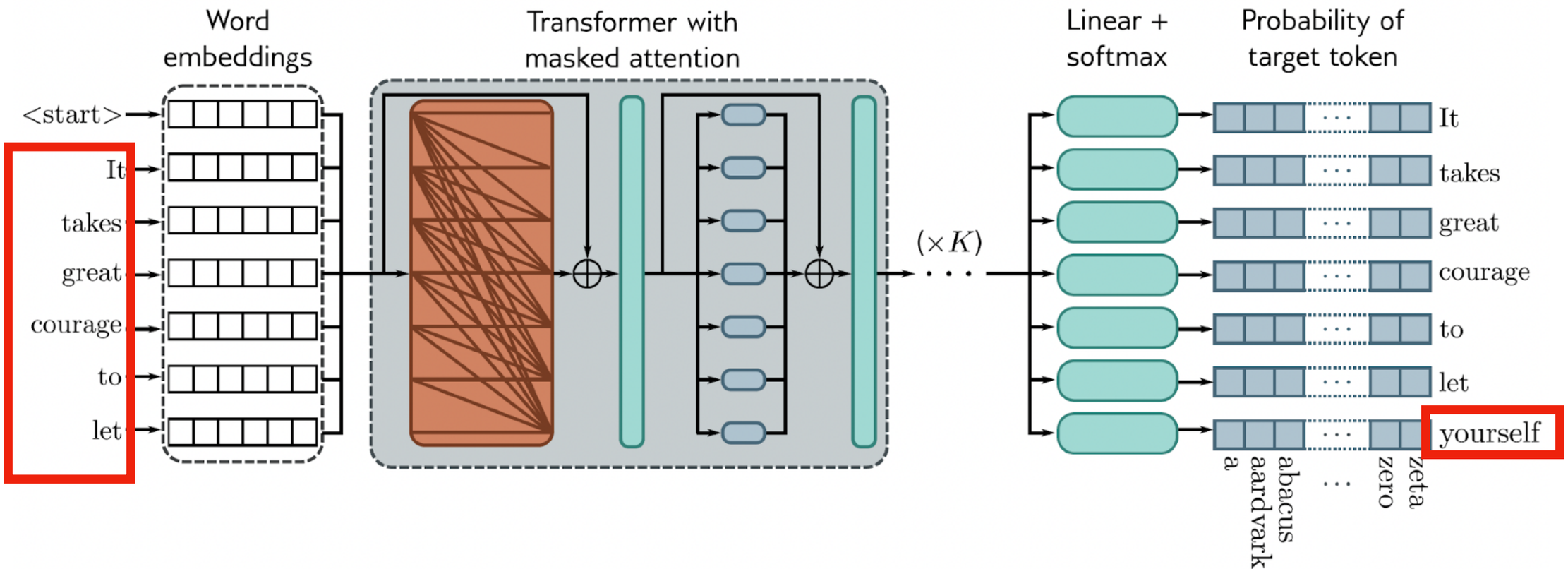
BERT

- Bidirectional Encoder Representations from Transformers (BERT)
  - Fine tunned to other NLP tasks



GPT-3

- Generative Pre-trained Transformer (GPT!)
- Decoder Only - predicts the next word!
- Large Language Model trained with larger and diverse set of data, capable of generating human like text.



GPT Family

- Large Language Models (LLMs) - models trained on vast amount of text data, based on the Transformer architecture and are noteworthy on language understanding, generative capabilities and interactive capabilities.

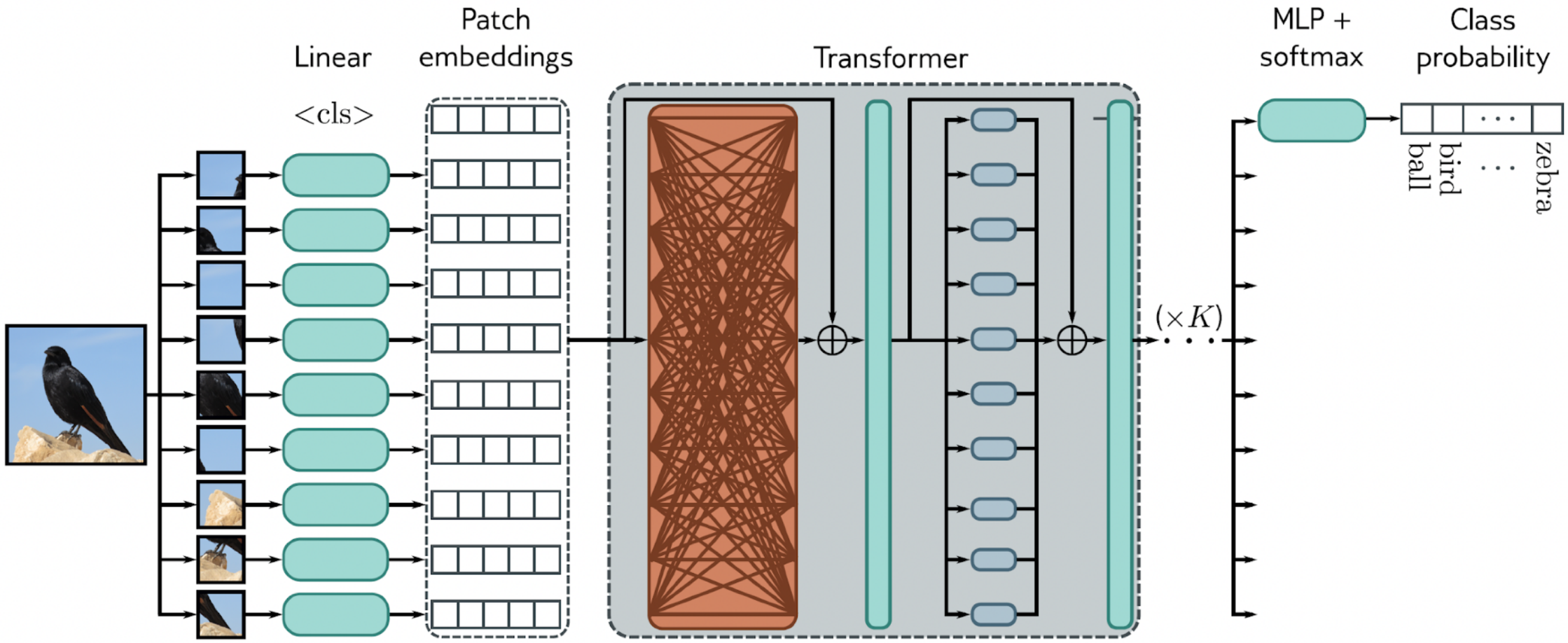
- GPT-X? Updates and Scale factors
  - 2018 - GPT: 117M
  - 2019 - GPT-2: 1.5B
  - 2020 - GPT-3: 175B
  - 2023 - GPT-4: 500B ?

Image GPT

- Autoregressive model - generates pixels by pixel depending on a conditional distribution

Vision Transformer (ViT)

- Vision transformer (ViT) - State of Art ?
  - CNNs destroyers?



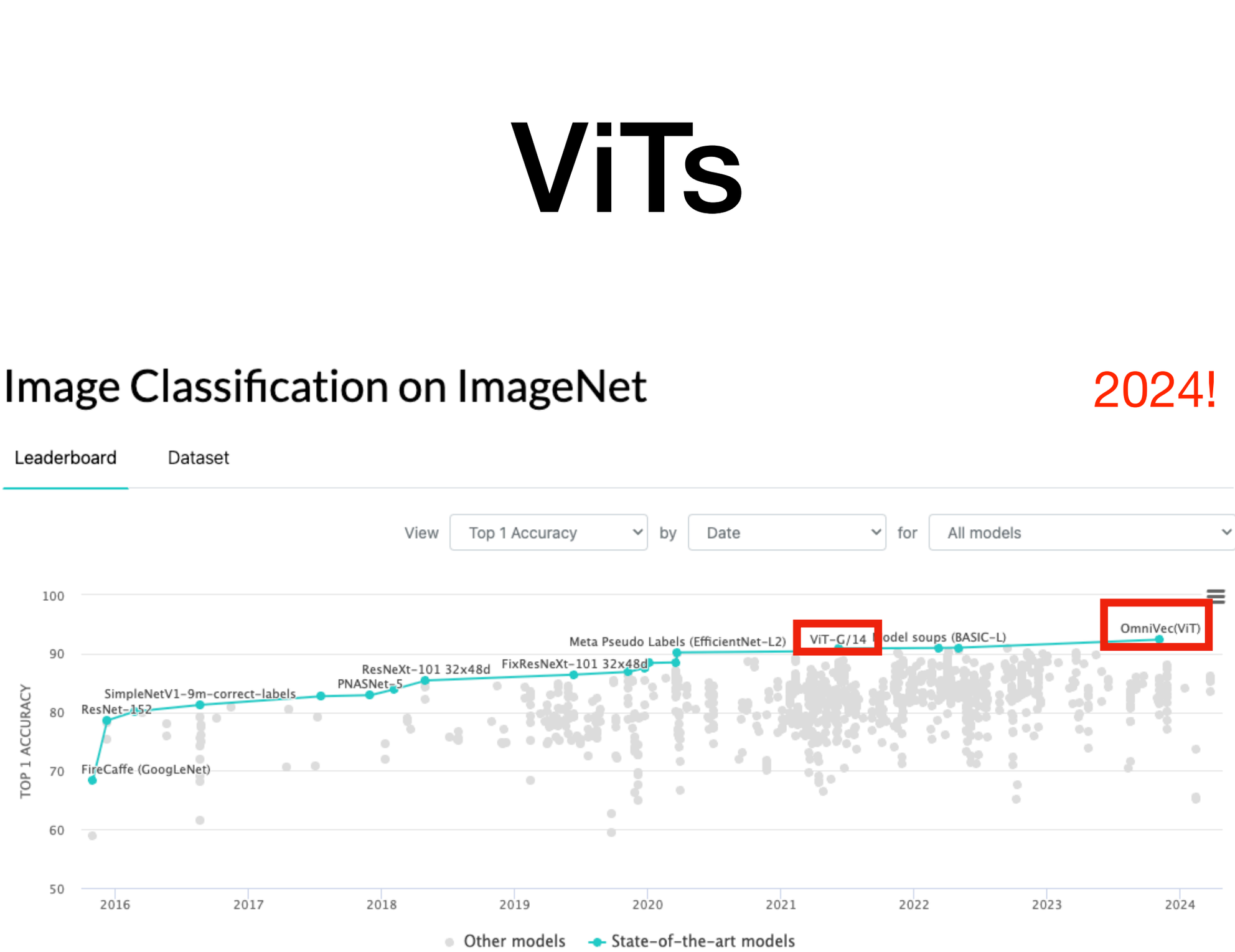
ViTs

Image Classification on ImageNet

2024!

Leaderboard Dataset

View Top 1 Accuracy by Date for All models



Filter: ImageNet-1k only | Transformer | ResNet | CNN | ImageNet-22k | EfficientNet | JFT-300M | MLP | ResNeXt | JFT-3B | Reversible | Neighborhood Attention | NAT Transformer | PatchConvnet | FPN | Conv+Transformer | ALIGN | MoE | Early Exit | Dynamic Model Arch | CNN+Transformer | CLIP data | SNN | IG-1B | Swin-Transformer | Teacher-22k | Vision Transformer | CrossCovarianceAttention | FLD-900M | Pure CNN | YFCC-15M | Laion-400M | Contrastive | ConvNeXt | Self-Supervised Learning | RegNet | Mixer | Memory-Centric | CLIP Pre-trained | untagged | Hardware Burden | Operations per network pass | Robustness reports

Edit Leaderboard

And More

Transformers
124,299

State-of-the-art ML for Pytorch, TensorFlow, and JAX.

Diffusers
22,331

State-of-the-art diffusion models for image and audio generation in PyTorch.

Safetensors
2,410

Simple, safe way to store and distribute neural networks weights safely and quickly.

Hub Python Library
1,646

Client library for the HF Hub: manage repositories from your Python runtime.

Tokenizers
8,378

Fast tokenizers, optimized for both research and production.

PEFT
13,612

Parameter efficient finetuning methods for large models.

Transformers.js
7,199

Community library to run pretrained models from Transformers in your browser.

timm
29,612

State-of-the-art computer vision models, layers, optimizers, training/evaluation, and utilities.

TRL
7,951

Train transformer language models with reinforcement learning.

Datasets
18,357

Access and share datasets for computer vision, audio, and

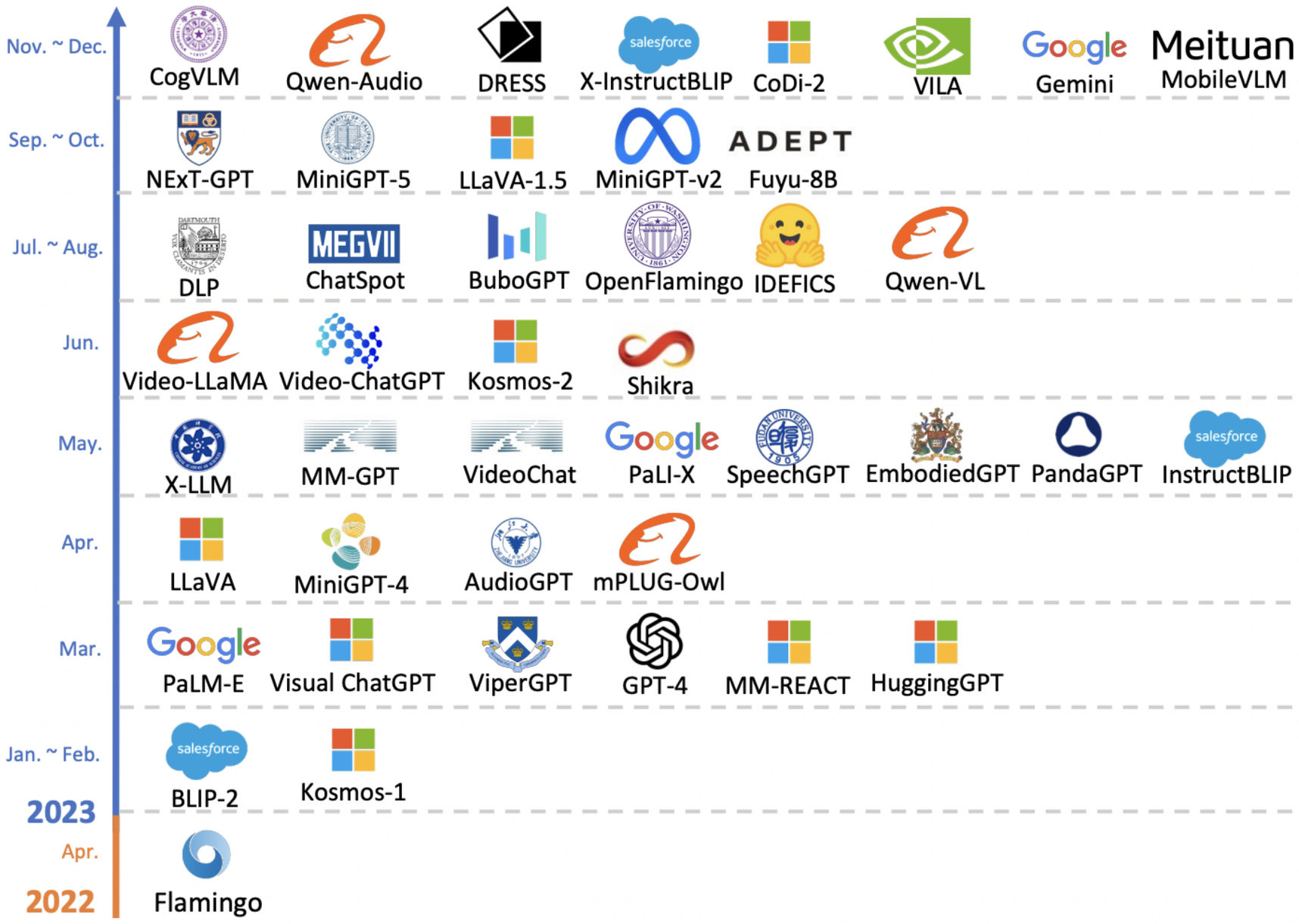
Text Generation Inference
7,733

Toolkit to serve Large Language Models.

Accelerate
6,894

Easily train and use PyTorch models with multi-GPU, TPU,

And More to come



And More to come

	Benchmark (Metric)	DeepSeek V3	DeepSeek V2.5 0905	Qwen2.5 72B-Inst	Llama3.1 405B-Inst	Claude-3.5 Sonnet-1022	GPT-4o 0513
	Architecture	MoE	MoE	Dense	Dense	-	-
	# Activated Params	37B	21B	72B	405B	-	-
	# Total Params	671B	236B	72B	405B	-	-
English	MMLU (EM)	88.5	80.6	85.3	**88.6**	88.3	87.2
	MMLU-Redux (EM)	**89.1**	80.3	85.6	86.2	88.9	88.0
	MMLU-Pro (EM)	75.9	66.2	71.6	73.3	**78.0**	72.6
	DROP (3-shot F1)	**91.6**	87.8	76.7	88.7	88.3	83.7
	IF-Eval (Prompt Strict)	86.1	80.6	84.1	86.0	**86.5**	84.3
	GPQA-Diamond (Pass@1)	59.1	41.3	49.0	51.1	**65.0**	49.9
	SimpleQA (Correct)	24.9	10.2	9.1	17.1	28.4	**38.2**
	FRAMES (Acc.)	73.3	65.4	69.8	70.0	72.5	**80.5**
	LongBench v2 (Acc.)	**48.7**	35.4	39.4	36.1	41.0	48.1
Code	HumanEval-Mul (Pass@1)	**82.6**	77.4	77.3	77.2	81.7	80.5
	LiveCodeBench (Pass@1-COT)	**40.5**	29.2	31.1	28.4	36.3	33.4
	LiveCodeBench (Pass@1)	**37.6**	28.4	28.7	30.1	32.8	34.2
	Codeforces (Percentile)	**51.6**	35.6	24.8	25.3	20.3	23.6
	SWE Verified (Resolved)	42.0	22.6	23.8	24.5	**50.8**	38.8
	Aider-Edit (Acc.)	79.7	71.6	65.4	63.9	**84.2**	72.9

References

- Christopher M. Bishop and Hugh Bishop, Deep Learning - Foundations and Concepts (2024). Springer 2024,
- Prince, S. J. (2023). Understanding Deep Learning. MIT Press.
- Drori, I. (2022). The Science of Deep Learning. Cambridge University Press.
- Kamath, U., Graham, K., & Emara, W. (Eds.). (2022). Transformers for Machine Learning. A Deep Dive. Chapman & Hall.